

山东大学

网络空间安全学院

《网络安全》

实验报告

学生姓名 江李阳 (学号:202400460042), 王健一 (学

号:202400460039), 邱泽辉 (学

号:202400460046)

指导教师 郭山清

学院 网络空间安全学院

专业班级 密码科学与技术 2024 级一班

完成时间 2026 年 9 月 10 日

1 实验目标

- 通过实验操作和网络流量分析，深入理解交换机和路由器的工作原理。
- 掌握子网划分和子网掩码的概念和应用，理解子网掩码在网络通信中的重要性。
- 熟练使用 tcpdump 工具，掌握获取和分析网络流量的技能。
- 为后续章节的学习和深入理解网络原理奠定基础，提升解决实际网络问题的能力。

2 实验原理

2.1 ARP

ARP(Address Resolution Protocol, 地址解析协议), 用于将 IP 地址解析为 MAC 地址。因为以太网通信时除了知道 IP 地址外, 还需要知道下一跳的 MAC 地址, 才能发送以太网帧。比如 A 与 B 通信时, A 希望给 B 发送以太网帧, 就必须知道 B 的 MAC 地址, 如果 A,B 不在同一个网段, 那么 A 需要知道网关的 MAC 地址。

当发送 IP 数据包时, 主机先查询自己的 ARP 缓存表, 如果已经有目标 IP 和 MAC 的映射, 则直接使用该映射发送以太网帧 (IP 数据包封装在以太网帧中)。如果没有, 则需要先构造 ARP 请求, 具体过程如下:

- ARP 请求时发送广播帧, 源 MAC 地址为 A 的 MAC 地址, 目的 MAC 地址为 FF:FF:FF:FF:FF:FF 即广播帧; 以太网类型为 0x0806 表示 ARP; 操作码为 1, 表示请求; 源 IP 地址为 A 的 IP 地址; ARP 报文中的目标硬件地址字段为全 0(因为此时还不知道目标 MAC 地址), 目的 IP 地址为要解析的 IP 地址, 如 B 的 IP 或网关 IP。
- 交换机收到广播帧后会向同一广播域内的其他端口泛洪, 因此同一局域网内的主机都会收到该 ARP 请求的广播帧, 但只有目标 IP 匹配的主机才会处理该请求, 其他主机直接丢弃。
- 目标主机 (或网关) 发现所问的就是自己时, 首先会更新自身的 ARP 缓存, 记录请求方的 IP 和 MAC 地址; 接着构造 ARP 响应, 操作码为 2, 该回复的以太网帧的目的 MAC 是请求方 MAC, 即以单播回复。
- A 收到 ARP 响应后将对应的下一跳 IP 与下一跳的 MAC 对应关系记录下来, 写入 ARP 缓存中。

2.2 ICMP

ICMP(Internet Control Message Protocol, 互联网控制报文协议) 属于网络层协议，但封装在 IP 包中，IPv4 首部中的协议号为 1 表示上层协议为 ICMP。它不是面向应用的数据传输协议，主要用于报告错误、网络诊断和路由控制等。常见类型包括：

- 查询类报文：Echo Request 的 Type 为 8，用于请求目标主机回应；Echo Reply 的 Type 为 0，用于回应 Echo Request。

差错类报文及其常见用途如下，其中 Redirect 报文会在子网掩码配置错误的实验中出现：

表 1 常见 ICMP 差错类报文

类型	名称	用途
3	Destination Unreachable	目标不可达
4	Source Quench	源抑制，已废弃
5	Redirect	重定向，告知主机有更好的下一跳
11	Time Exceeded	超时，TTL 减到 0 或分片重组超时
12	Parameter Problem	IP 头部字段问题

Type 3 和 Type 11 报文中常见的 Code 字段含义如下：

表 2 ICMP Type 3 的常见 Code

Code	含义
0	网络不可达
1	主机不可达
2	协议不可达
3	端口不可达
4	需要分片但 DF 位置位
5	源路由失败

2.3 Ping

Ping 的本质是发送 ICMP Echo Request，等待对方回复 ICMP Echo Reply，计算往返时间 RTT。Ping 并不使用 TCP/UDP，因此没有端口，而是直接使用 ICMP。Ping 的完整流程如下 (假设 A Ping B):

表 3 ICMP Type 11 的常见 Code

Code	含义
0	TTL 超时
1	分片重组超时

- 解析目标 IP: A 执行 `ping test.com`, 此时 Ping 的目标如果是域名, 系统先会做 DNS 解析, 得到对应的 IP; 如果 Ping 的是 IP, 则直接使用。
- 构造 ICMP 报文 (Type=8, 类型为 Echo Request): 封装进 IP 包, 加上 IP 头部信息, 包括源 IP(A 的 IP 地址) 和目的 IP(B 的 IP 地址), 协议号为 1 表示 ICMP。
- 封装 IP 包: 发送前必须知道下一跳 MAC, 如果是 B 在相同子网下, 下一跳就是 B, ARP 解析出 B 的 MAC 地址 (查 ARP 缓存或构造 ARP 请求); 如果 B 与 A 不在同一子网, 即跨网段了, 则 ARP 解析网关的 MAC 地址。最终封装成以太网帧并发出。
- 网络转发: 若 B 与 A 在相同子网, 则由交换机根据 MAC 和端口映射关系直接转发; 若不在同一子网, 则由路由器转发, 经过路由器时会去掉原来的以太网帧头, 将其中的 IP 包部分字段做修改 (如 TTL 值减一、重新计算校验和等), 查询路由表确认下一跳 IP 地址以及对应 MAC 地址, 重新封装以太网帧 (如果 TTL 减为 0, 路由器直接丢包, 并返回 ICMP Time Exceeded)。
- 收到 ICMP 请求: 当 B 收到 IP 包后, 检查发现目的 IP 的确是自身且协议是 ICMP(类型 8, Echo Request), 然后 B 将构造 Echo Reply 并将该回复发送给 A(该过程同样要经历查 ARP 并发送给下一跳)。
- 收到 ICMP 回复: 当 A 收到 Echo Reply 后会先检查 ICMP 校验和、检查序号等参数与之前的请求相匹配, 然后用收到时间减去发送时的时间计算 RTT。

3 实验设置

实验步骤、网络拓扑结构等符合实验指导书的要求, 但具体每人实验实验时使用的网卡、IP 地址、MAC 地址等并不完全一致, 具体每个实验前会详述本实验设置。

4 实验一：使用 tcpdump 分析 ICMP 流量

4.1 实验配置

本实验使用 VMware 搭建实验环境，Client(Attacker) 通过 OpenWRT 路由器与其他主机进行通信。Client 使用的网卡为 `ens33`，其地址为 `192.168.3.10`，OpenWRT 内网接口地址为 `192.168.3.1`，Victim 地址为 `192.168.3.200`，External Server 地址为 `192.168.4.105`。实验开始时 Client 和 Victim 使用 `/24` 掩码，第三部分实验再将 Client 的掩码修改为 `/28`。

在 Client 上使用如下命令清空 ARP 缓存并捕获 ARP 和 ICMP 报文：

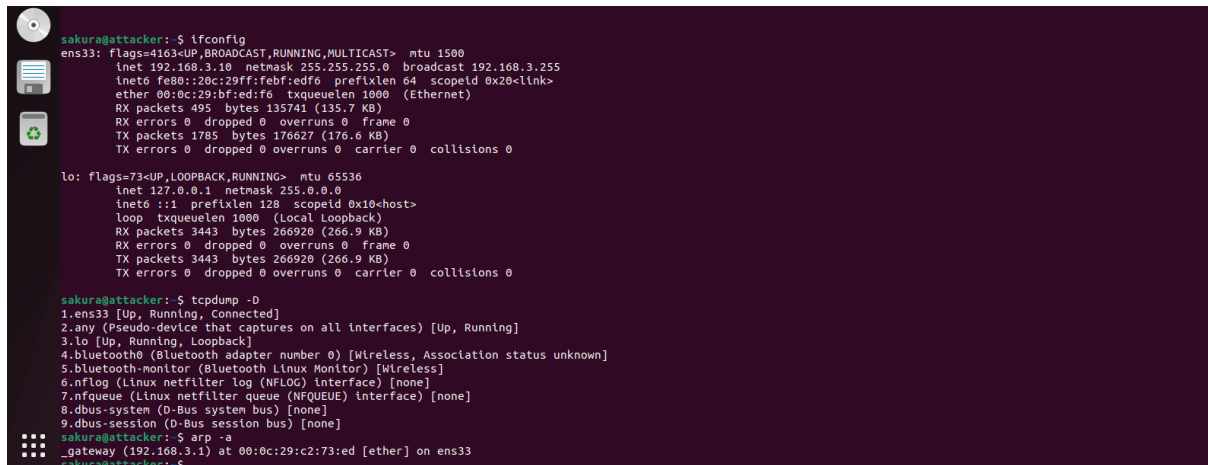
Listing 1 Client 上的 tcpdump 抓包命令

```
1 sudo ip neigh flush dev ens33
2 sudo tcpdump -i ens33 -n -e 'arp or icmp'
```

其中，`-i ens33` 表示在 `ens33` 网卡上抓包，`-n` 表示不进行地址解析，`-e` 表示显示以太网帧头中的 MAC 地址。

4.2 子网内通信

首先登录 Client，查看当前的网络接口、可监听的网卡、ARP 缓存表等如下图：



```
sakura@attacker:~$ ifconfig
ens33: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 192.168.3.10 netmask 255.255.255.0 broadcast 192.168.3.255
    inet6 fe80::20c:29ff:feb1:edf6 prefixlen 64 scopeid 0x20<link>
    ether 00:0c:29:b1:ed:f6 txqueuelen 1000 (Ethernet)
    RX packets 495 bytes 135741 (135.7 KB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 1785 bytes 176627 (176.6 KB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
    inet 127.0.0.1 netmask 255.0.0.0
    inet6 ::1 prefixlen 128 scopeid 0x10<host>
    loop txqueuelen 1000 (Local Loopback)
    RX packets 3443 bytes 266920 (266.9 KB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 3443 bytes 266920 (266.9 KB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

sakura@attacker:~$ tcpdump -D
1.ens33 [Up, Running, Connected]
2.any (Pseudo-device that captures on all interfaces) [Up, Running]
3.lo [Up, Running, Loopback]
4.bluetooth0 (Bluetooth adapter number 0) [Wireless, Association status unknown]
5.bluetooth-monitor (Bluetooth Linux Monitor) [Wireless]
6.nflog (Linux netfilter log (NFLOG) interface) [none]
7.nfqueue (Linux netfilter queue (NFQUEUE) interface) [none]
8.dbus-system (D-Bus system bus) [none]
9.dbus-session (D-Bus session bus) [none]

sakura@attacker:~$ arp -a
_gateway (192.168.3.1) at 00:0c:29:c2:73:ed [ether] on ens33
sakura@attacker:~$
```

图 1 client 的网络信息

然后在 Client 上清空 ARP 缓存，在另一个终端执行（注意此前 Client 已经开启监听 `ens33` 网卡）：

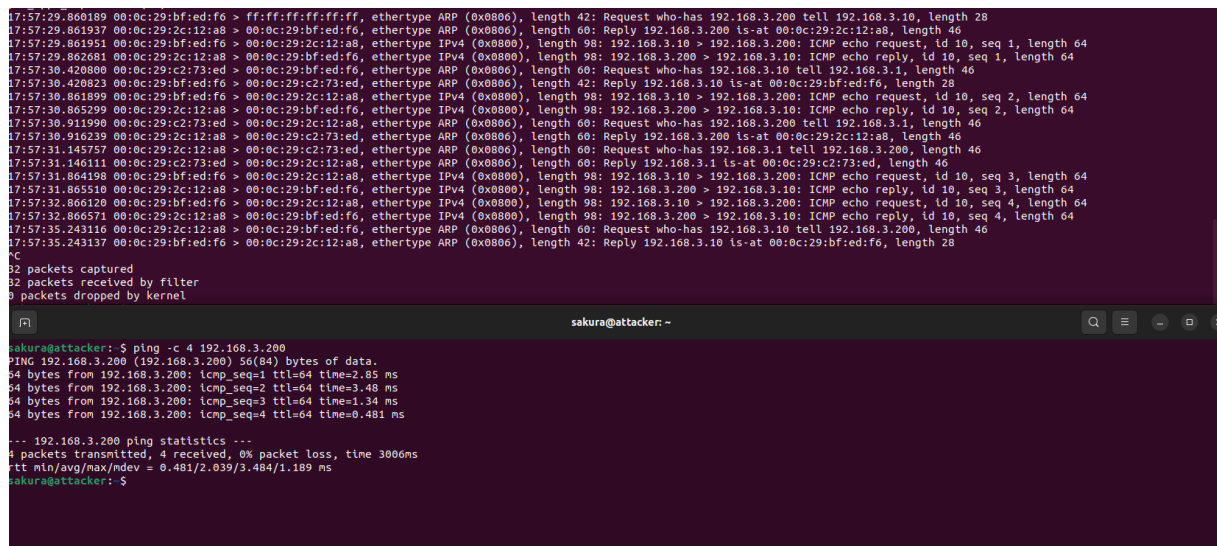
Listing 2 子网内通信测试

```
1 ping -c 4 192.168.3.200
```

由于 `192.168.3.10` 和 `192.168.3.200` 均属于 `192.168.3.0/24`，Client 判断目标主机与自己处于同一子网内，因此不会先将数据发送给网关，而是直接广播 ARP 请求，询问

192.168.3.200 对应的 MAC 地址。Victim 收到请求后单播返回 ARP 响应，Client 随后将 ICMP Echo Request 直接发送给 Victim。

从抓包结果可以看到，ARP 请求的源 MAC 地址为 Client 的 MAC 地址 00:0c:29:bf:ed:f6，目的以太网 MAC 地址为广播地址；ARP 响应的源 MAC 地址为 Victim 的 MAC 地址 00:0c:29:2c:12:a8。之后出现源 IP 为 192.168.3.10、目的 IP 为 192.168.3.200 的 ICMP Echo Request，以及方向相反的 ICMP Echo Reply。



```
17:57:29.860189 00:0c:29:bf:ed:f6 > ff:ff:ff:ff:ff:ff, ethertype ARP (0x0806), length 42: Request who-has 192.168.3.200 tell 192.168.3.10, length 28
17:57:29.861937 00:0c:29:2c:12:a8 > 00:0c:29:bf:ed:f6, ethertype ARP (0x0806), length 60: Reply 192.168.3.200 is-at 00:0c:29:2c:12:a8, length 46
17:57:29.861951 00:0c:29:bf:ed:f6 > 00:0c:29:2c:12:a8, ethertype IPv4 (0x0800), length 98: 192.168.3.10 > 192.168.3.200: ICMP echo request, id 10, seq 1, length 64
17:57:29.862681 00:0c:29:2c:12:a8 > 00:0c:29:bf:ed:f6, ethertype IPv4 (0x0800), length 98: 192.168.3.200 > 192.168.3.10: ICMP echo reply, id 10, seq 1, length 64
17:57:30.420800 00:0c:29:c2:73:ed > 00:0c:29:bf:ed:f6, ethertype ARP (0x0806), length 60: Request who-has 192.168.3.10 tell 192.168.3.1, length 46
17:57:30.420823 00:0c:29:bf:ed:f6 > 00:0c:29:c2:73:ed, ethertype ARP (0x0806), length 42: Reply 192.168.3.10 is-at 00:0c:29:bf:ed:f6, length 28
17:57:30.861899 00:0c:29:bf:ed:f6 > 00:0c:29:2c:12:a8, ethertype IPv4 (0x0800), length 98: 192.168.3.10 > 192.168.3.200: ICMP echo request, id 10, seq 2, length 64
17:57:30.865299 00:0c:29:2c:12:a8 > 00:0c:29:bf:ed:f6, ethertype IPv4 (0x0800), length 98: 192.168.3.200 > 192.168.3.10: ICMP echo reply, id 10, seq 2, length 64
17:57:30.911990 00:0c:29:c2:73:ed > 00:0c:29:2c:12:a8, ethertype ARP (0x0806), length 60: Request who-has 192.168.3.200 tell 192.168.3.1, length 46
17:57:30.916239 00:0c:29:2c:12:a8 > 00:0c:29:c2:73:ed, ethertype ARP (0x0806), length 60: Reply 192.168.3.200 is-at 00:0c:29:2c:12:a8, length 46
17:57:31.145757 00:0c:29:2c:12:a8 > 00:0c:29:c2:73:ed, ethertype ARP (0x0806), length 60: Request who-has 192.168.3.1 tell 192.168.3.200, length 46
17:57:31.146111 00:0c:29:c2:73:ed > 00:0c:29:2c:12:a8, ethertype ARP (0x0806), length 60: Reply 192.168.3.1 is-at 00:0c:29:c2:73:ed, length 46
17:57:31.864198 00:0c:29:bf:ed:f6 > 00:0c:29:2c:12:a8, ethertype IPv4 (0x0800), length 98: 192.168.3.10 > 192.168.3.200: ICMP echo request, id 10, seq 3, length 64
17:57:31.865510 00:0c:29:2c:12:a8 > 00:0c:29:bf:ed:f6, ethertype IPv4 (0x0800), length 98: 192.168.3.200 > 192.168.3.10: ICMP echo reply, id 10, seq 3, length 64
17:57:32.866120 00:0c:29:bf:ed:f6 > 00:0c:29:2c:12:a8, ethertype IPv4 (0x0800), length 98: 192.168.3.10 > 192.168.3.200: ICMP echo request, id 10, seq 4, length 64
17:57:32.866571 00:0c:29:2c:12:a8 > 00:0c:29:bf:ed:f6, ethertype IPv4 (0x0800), length 98: 192.168.3.200 > 192.168.3.10: ICMP echo reply, id 10, seq 4, length 64
17:57:35.243116 00:0c:29:2c:12:a8 > 00:0c:29:bf:ed:f6, ethertype ARP (0x0806), length 60: Request who-has 192.168.3.10 tell 192.168.3.200, length 46
17:57:35.243137 00:0c:29:bf:ed:f6 > 00:0c:29:2c:12:a8, ethertype ARP (0x0806), length 42: Reply 192.168.3.10 is-at 00:0c:29:bf:ed:f6, length 28
^C
32 packets captured
32 packets received by filter
0 packets dropped by kernel

sakura@attacker: ~
sakura@attacker:~$ ping -c 4 192.168.3.200
PING 192.168.3.200 (192.168.3.200) 56(84) bytes of data:
64 bytes from 192.168.3.200: icmp_seq=1 ttl=64 time=2.85 ms
64 bytes from 192.168.3.200: icmp_seq=2 ttl=64 time=3.48 ms
64 bytes from 192.168.3.200: icmp_seq=3 ttl=64 time=1.34 ms
64 bytes from 192.168.3.200: icmp_seq=4 ttl=64 time=0.481 ms

--- 192.168.3.200 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3006ms
rtt min/avg/max/mdev = 0.481/2.039/3.484/1.189 ms
sakura@attacker:~$
```

图 2 子网内 Client 与 Victim 通信的 tcpdump 抓包结果

4.3 跨子网通信

在保持 Client 地址不变的情况下，清空 ARP 缓存后访问 External Server：

Listing 3 跨子网通信测试

```
1 sudo ip neigh flush dev ens33
2 ping -c 4 192.168.4.105
```

192.168.4.105 不属于 Client 所在的 192.168.3.0/24 网段，因此 Client 需要将数据包交给默认网关 192.168.3.1。Client 首先通过 ARP 请求获取网关的 MAC 地址，OpenWRT 返回 ARP 响应。之后，Client 发送的以太网帧目的 MAC 地址是 OpenWRT 内网接口的 MAC 地址 00:0c:29:c2:73:ed，但 IP 层的目的地址仍然是 External Server 的 192.168.4.105。

在 Client 的 ens33 网卡上抓包时，看到的跨网段 ICMP 报文为 192.168.3.10 与 192.168.4.105 之间的 Echo Request 和 Echo Reply，而以太网帧的下一跳 MAC 地址是 OpenWRT 的 MAC 地址。这说明路由器负责三层转发，并在不同网段重新封装以太网帧。

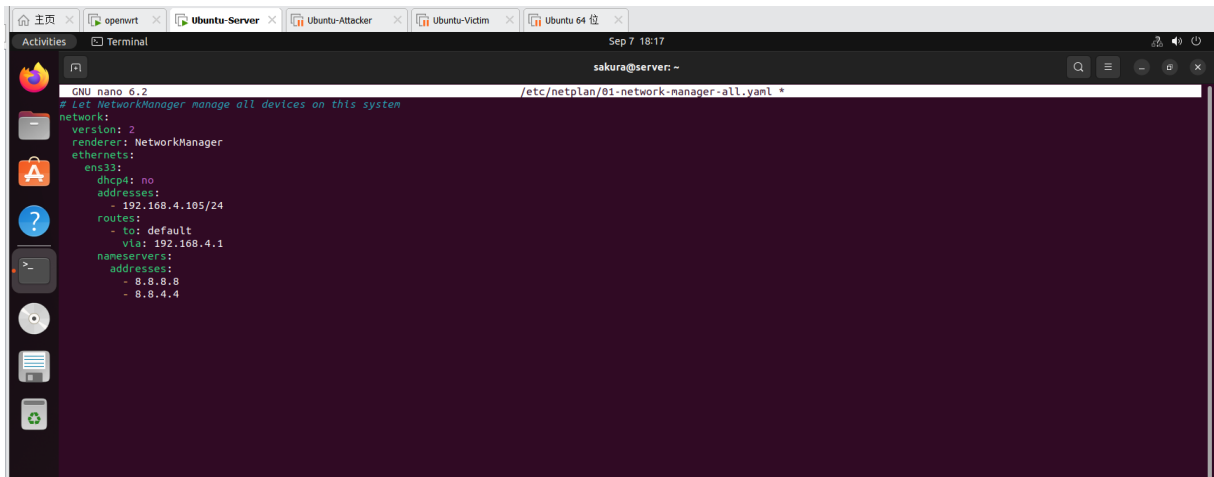


图 3 External Server 的网络配置

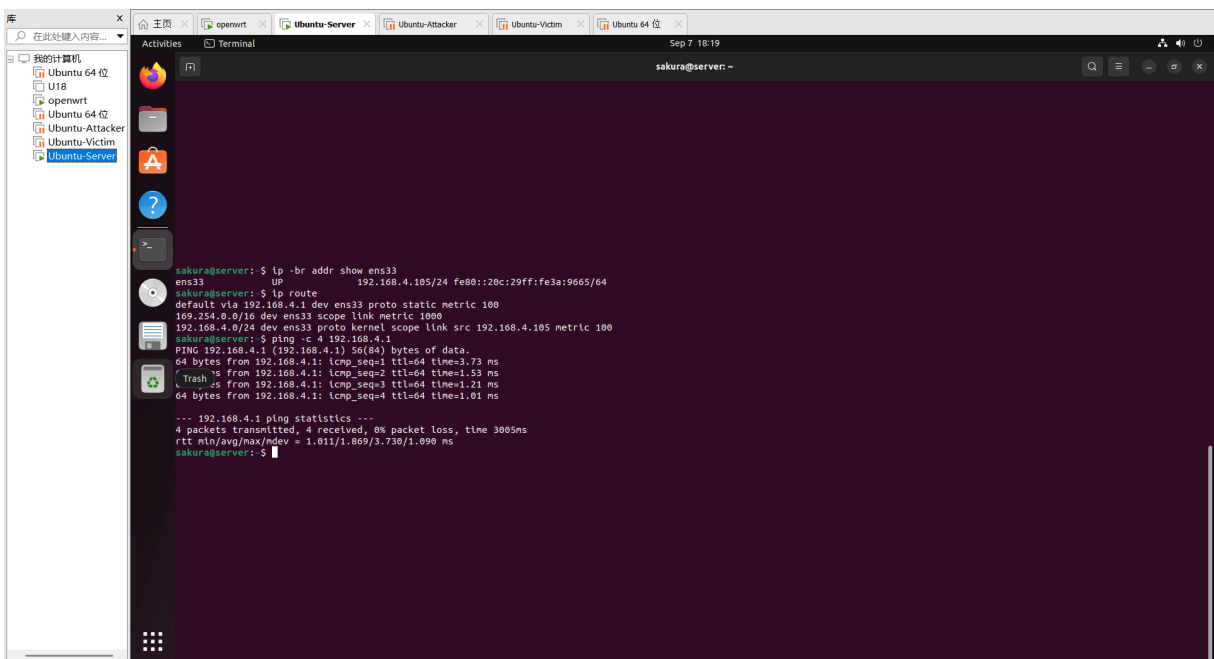


图 4 跨子网通信前的配置验证

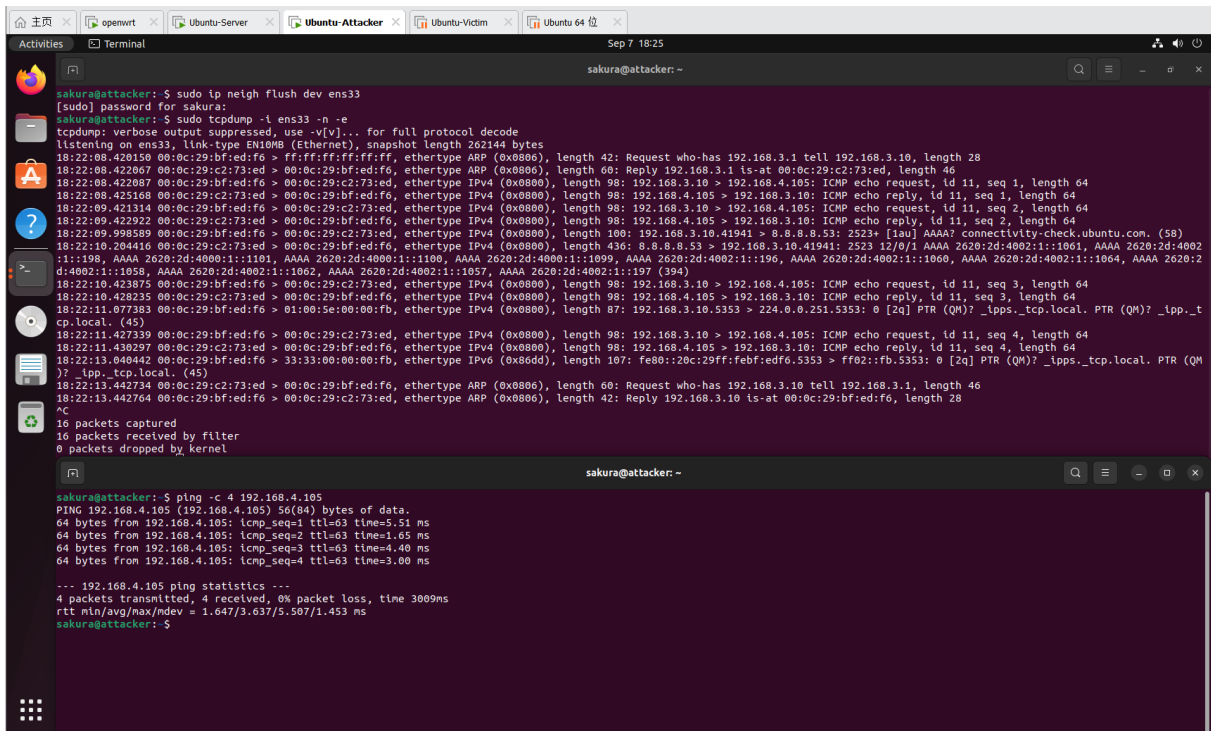


图 5 Client 访问 External Server 的实验结果

4.4 子网掩码错误

第三部分实验按照实验指导书的要求，将 Client 的子网掩码由/24 修改为/28。修改后 Client 的地址和路由信息为：

Listing 4 修改 Client 子网掩码并验证

```
1 sudo nano /etc/netplan/01-network-manager-all.yaml
2 sudo netplan generate
3 sudo netplan try
4 ip -br addr show ens33
5 ip route
```

修改后，Client 的地址变为 192.168.3.10/28，直连网段为 192.168.3.0/28。虽然 Victim 的地址仍然是 192.168.3.200，并且二者实际上连接在同一个二层网络中，但 Client 根据/28 掩码判断 192.168.3.200 不属于本地子网，因此首先将 ICMP 报文发送给默认网关 192.168.3.1。

由于 OpenWRT 发现目标 192.168.3.200 与报文进入的接口相同，路由器向 Client 返回了 ICMP Redirect 报文，并提示新的下一跳为 192.168.3.200。Client 收到重定向后重新进行 ARP 解析，获取 Victim 的 MAC 地址，之后直接向 Victim 发送 ICMP Echo Request。因此，抓包中依次可以观察到：Client 与网关之间的 ARP 请求和响应、ICMP Redirect、Client 对 Victim 的 ARP 请求和响应，以及最终的 ICMP Echo Request 和

Echo Reply。

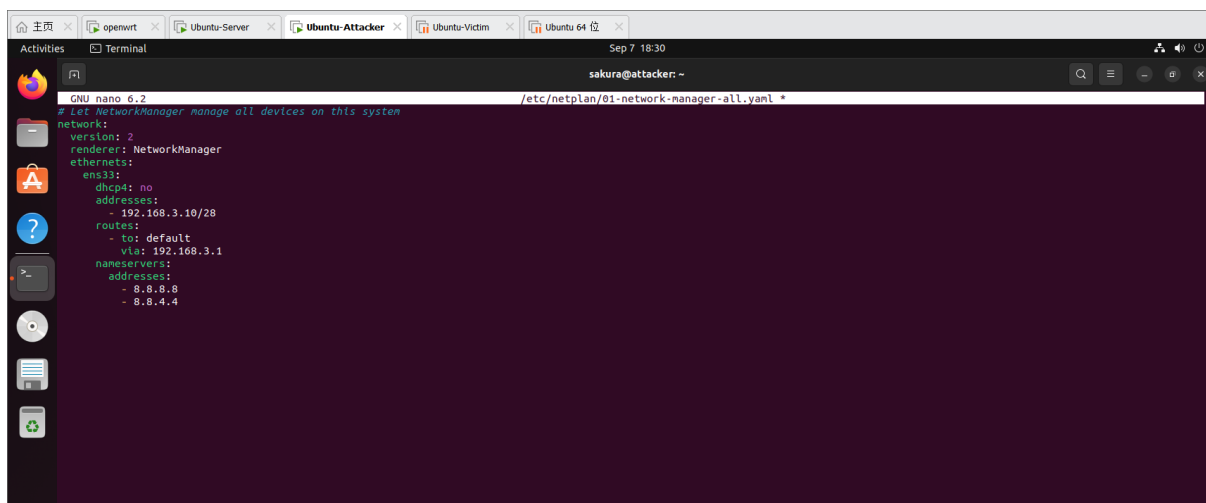


图 6 修改 Client 子网掩码为/28

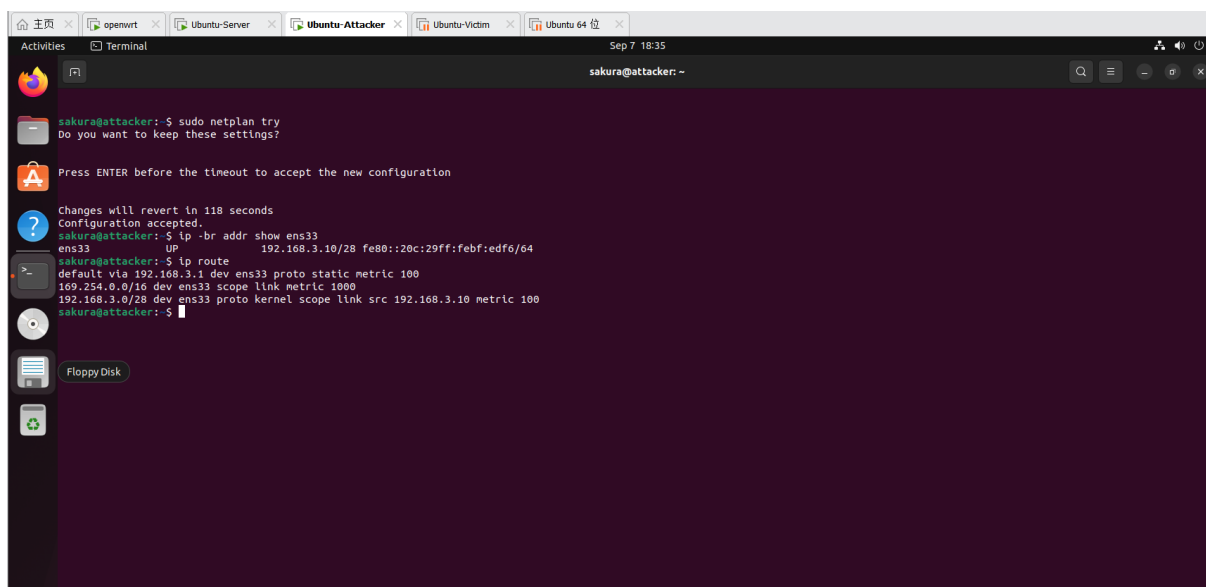


图 7 /28 掩码配置验证

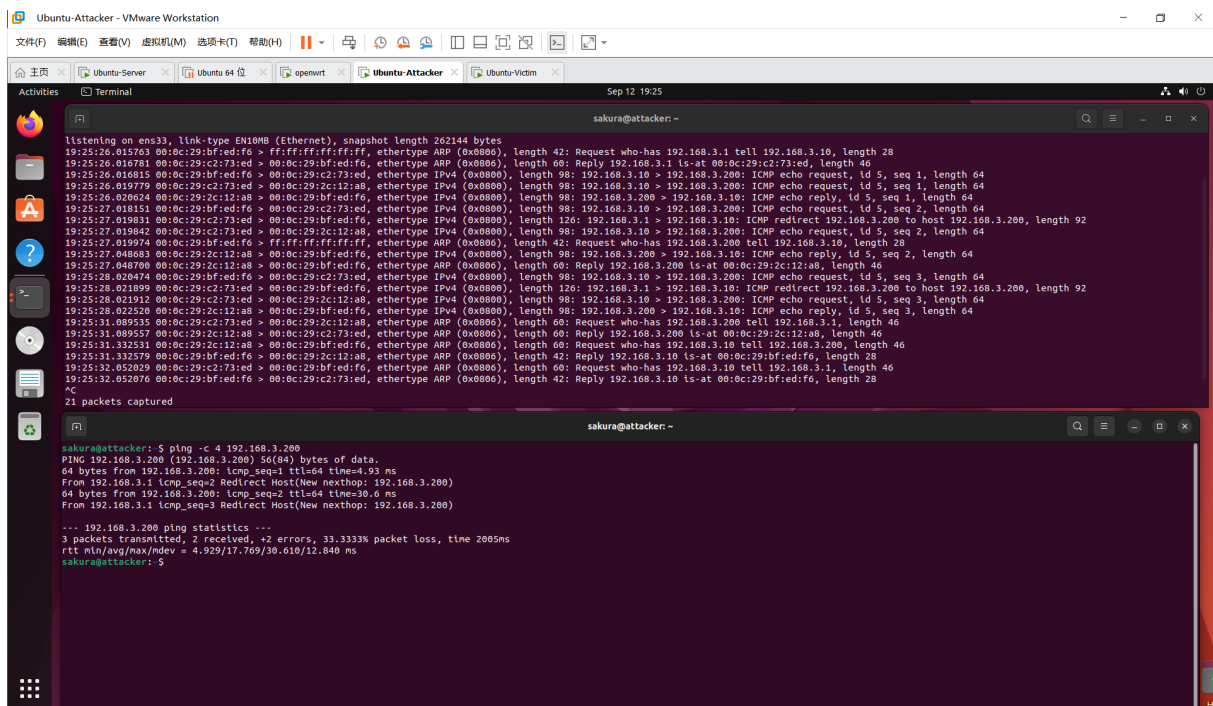


图 8 子网掩码错误时的 tcpdump 与 ping 结果

从图中可以看到，ping 过程中出现了“Redirect Host”提示，同时也收到了 Victim 的 Echo Reply。终端最后显示 3 个数据包中收到 2 个回复，并统计了 2 个错误。这里的错误是 Linux 的 ping 程序将 ICMP Redirect 作为错误报文统计的结果，并不表示 Victim 完全不可达；tcpdump 已经证明重定向前后均存在 ICMP Echo Reply。

4.5 流量分析记录表

表4按照实验指导书中的表 1-1-1 填写。表中的 MAC 地址是从 Client 的 ens33 网卡抓包结果中读取的；因此在跨子网通信中，Client 看到的下一跳 MAC 是 OpenWRT 内网接口的 MAC，而不是 External Server 在另一个网段中的 MAC。

表 4 实验一流量分析记录表

操作	ARP 请求		ARP 响应		ICMP ECHO 请求		ICMP ECHO 响应	
	源 MAC	目标 MAC	源 MAC	目标 MAC	源 IP	目标 IP	源 IP	目标 IP
子网内	00:0c:29:bf:ed:f6	ff:ff:ff:ff:ff:ff	00:0c:29:2c:12:a8	00:0c:29:bf:ed:f6	192.168.3.10	192.168.3.200	192.168.3.200	192.168.3.10
跨子网	00:0c:29:bf:ed:f6	ff:ff:ff:ff:ff:ff	00:0c:29:c2:73:ed	00:0c:29:bf:ed:f6	192.168.3.10	192.168.4.105	192.168.4.105	192.168.3.10
掩码错（初始）	00:0c:29:bf:ed:f6	ff:ff:ff:ff:ff:ff	00:0c:29:c2:73:ed	00:0c:29:bf:ed:f6	192.168.3.10	192.168.3.200	192.168.3.200	192.168.3.10
掩码错（重定向后）	00:0c:29:bf:ed:f6	ff:ff:ff:ff:ff:ff	00:0c:29:2c:12:a8	00:0c:29:bf:ed:f6	192.168.3.10	192.168.3.200	192.168.3.200	192.168.3.10

根据实际的 Ping 结果应当指出，“掩码错”对应两个阶段：初始阶段 Client 向网关 192.168.3.1 发出 ARP 请求，ARP 响应来自 OpenWRT；收到 ICMP Redirect 后，Client 再次发出 ARP 请求，ARP 响应来自 Victim。两个阶段的 ICMP 源、目的 IP 仍然是

192.168.3.10 和 192.168.3.200，变化的是以太网帧的下一跳 MAC 地址。

4.6 三次操作的结果差异及原因分析

根据表4，三次操作的主要差异在于 ARP 解析的对象、以太网帧的下一跳 MAC 地址以及是否经过路由器转发，具体原因是 Client 的子网掩码改变了它对目标地址所属网络的判断。

- 子网内通信时，Client 和 Victim 都属于 192.168.3.0/24。Client 直接 ARP 解析 Victim 的 MAC 地址，之后以太网帧由交换机直接转发，ICMP Echo Request 和 Echo Reply 的 IP 地址分别为 192.168.3.10 和 192.168.3.200，过程中不需要经过 OpenWRT 进行三层路由。
- 跨子网通信时，External Server 不属于 Client 所在的 192.168.3.0/24 网段。Client 不能直接解析 External Server 的 MAC 地址，而是 ARP 解析默认网关 192.168.3.1 的 MAC 地址，由 OpenWRT 进行三层转发。因此在 Client 的网卡上，IP 层的目的地址是 External Server 的地址，但以太网帧的目的 MAC 地址是 OpenWRT 内网接口的 MAC 地址。
- 子网掩码错误时，Client 使用 192.168.3.10/28，将 192.168.3.200 错误判断为远端地址，所以首先 ARP 解析网关并把 ICMP 报文发送给 OpenWRT。OpenWRT 发现目标与报文进入同一个内网接口，于是返回 ICMP Redirect，并告知 Client 可以把 192.168.3.200 作为新的下一跳。Client 随后重新 ARP 解析 Victim 并直接发送数据。因此，该操作同时出现了网关 ARP、ICMP Redirect 和 Victim ARP，形成了与前两次操作不同的报文序列。

由此可见，三次操作中 ICMP 报文的 IP 通信端点并不一定改变，但子网掩码会影响主机的路由判断，进而决定 ARP 请求的目标、以太网帧的下一跳以及是否触发路由器的重定向。

4.7 实验小结

本实验通过 tcpdump 观察了 ARP 和 ICMP 报文在不同网络配置下的变化。子网内通信时，Client 直接解析 Victim 的 MAC 地址并发送数据；跨子网通信时，Client 解析默认网关的 MAC 地址，由 OpenWRT 完成三层转发；子网掩码错误时，Client 错误地把同一二层网络中的 Victim 判断为远端主机，先将数据交给网关，并收到 ICMP Redirect。由此可以看出，子网掩码不仅决定主机对目标地址的判断，也会直接影响 ARP 解析对象、下一跳选择和数据包的转发过程。

5 实验二：使用 Wireshark 分析 Web 访问过程流量

5.1 实验目的

1. 学习并掌握 Wireshark 在网络流量分析中的作用，以及如何通过它来捕获和分析网络数据包；
2. 通过实际操作，观察并分析域名解析过程中涉及的 IP 包，理解 DNS 请求和响应的格式和内容，以及它是如何将域名转换为 IP 地址的；
3. 观察并解释 Client 与 www.imool.com.cn 之间 TCP 连接的建立和拆除过程，包括三次握手和四次挥手；
4. 使用 Wireshark 的协议流追踪功能，提取并分析 Web 访问过程中产生的 Cookie 信息

5.2 实验配置

本次实验采用 Oracle VirtualBox 搭建实验环境，OpenWRT 路由器通过三个接口分别和 Client、Web Server、DNS 相连，具体网络拓扑的配置如下：

表 5 实验二各虚拟机网络接口及 IP 地址配置

虚拟机	Linux 内网卡名	IP 地址	默认网关
Client	enp0s3	192.168.3.10/24	192.168.3.1
DNS Server	enp0s3	192.168.2.53/24	192.168.2.1
Web Server	enp0s3	192.168.4.105/24	192.168.4.1
OpenWRT	eth1	192.168.4.1/24	—
OpenWRT	eth2	192.168.3.1/24	—
OpenWRT	eth3	192.168.2.1/24	—
OpenWRT	eth0	DHCP	—

网络拓扑如图所示：

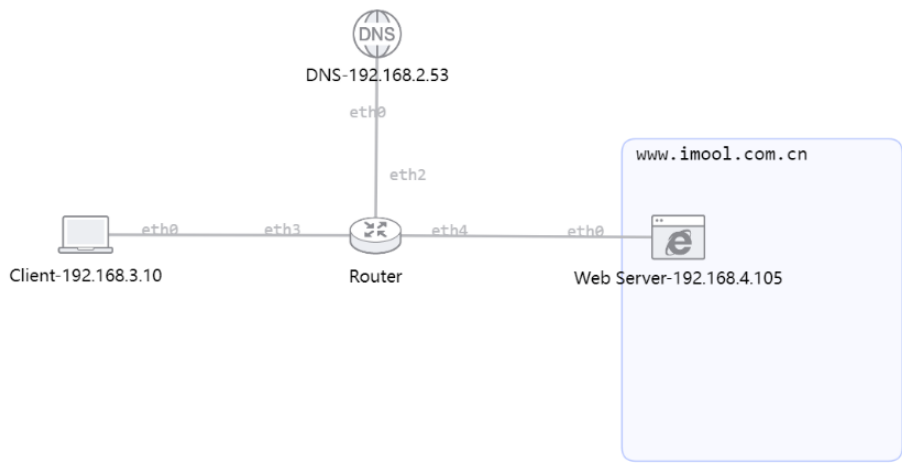


图 1-2-1 实验拓扑图

图 9 实验二网络拓扑图

5.3 实验过程

5.3.1 配置 Web Server

在 Web Server 主机 192.168.4.105 上启动 HTTP 服务。由于系统中未安装 Apache 或 Nginx，因此使用 Python 自带的 HTTP Server 建立简单的 Web 服务。执行：

Listing 5 建立简单的 Web 服务

```
1 sudo python3 -m http.server 80 --bind 0.0.0.0
2 ss -lntp
```

查看端口监听状态，可以看到 0.0.0.0:80 已处于监听状态，说明 Web Server 已成功提供 HTTP 服务。

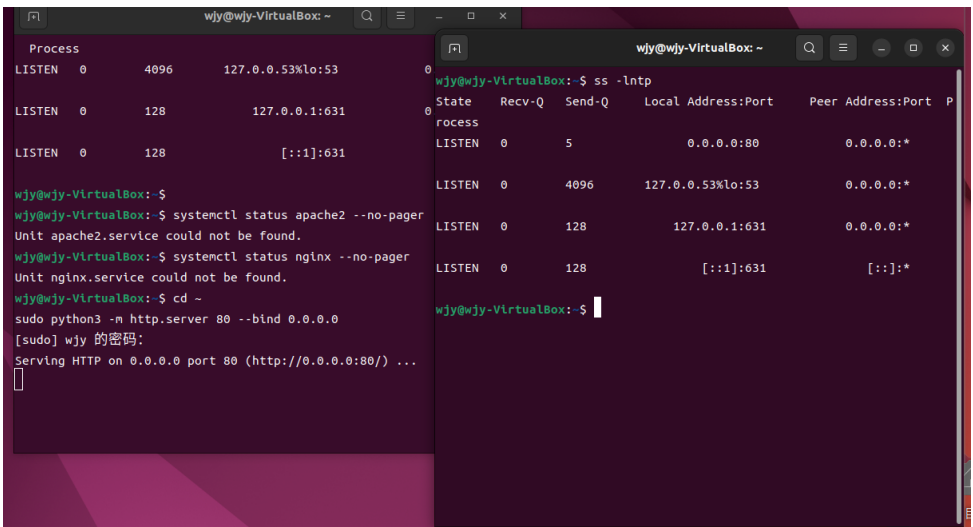


图 10 Web Server 启动 HTTP 服务截图

5.3.2 配置 DNS Server

在虚拟机 DNS Server 上打开终端，使用 nano 编辑器创建配置文件：

Listing 6 创建 DNS 配置文件

```
1 cat > ~/imool-dnsmasq.conf <<'EOF'
2 listen-address=192.168.2.53
3 bind-interfaces
4 address=/www.imool.com.cn/192.168.4.105
5 no-resolv
6 EOF
```

检查配置文件的语法，如下：

Listing 7 检查 DNS 配置文件语法

```
1 sudo dnsmasq --test --conf-file="$HOME/imool-dnsmasq.conf"
```

在确认后，启用 DNS 服务，如下：

Listing 8 启用 DNS 服务

```
1 sudo dnsmasq --no-daemon --conf-file="$HOME/imool-dnsmasq.conf"
```

此时 DNS Server 可以响应 Client 对 www.imool.com.cn 的解析请求，并返回 Web Server 的 IP 地址 192.168.4.105。

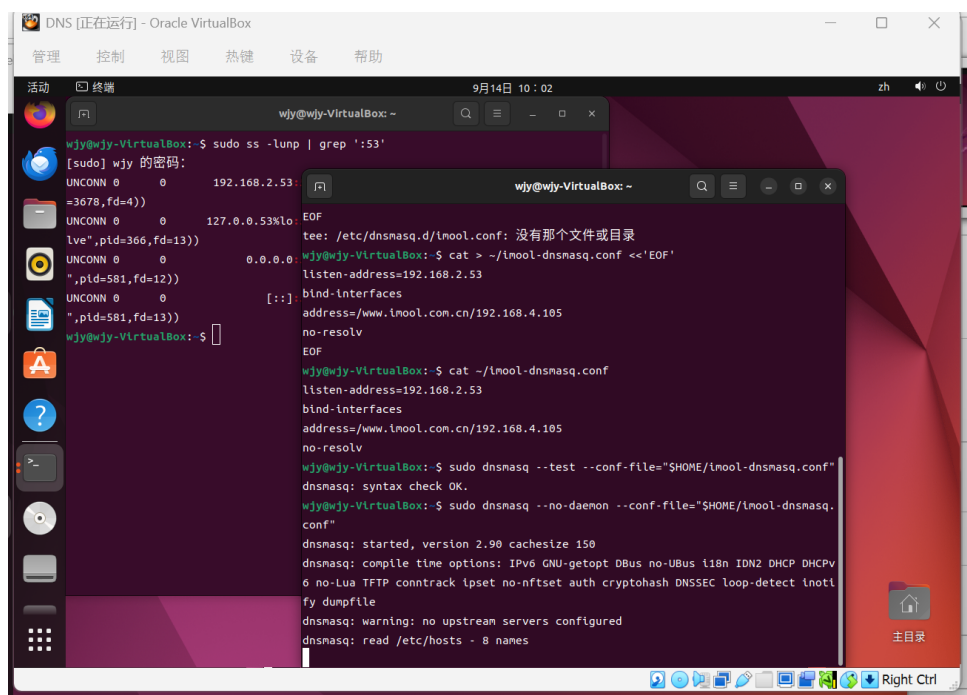


图 11 DNS Server 启动截图

5.3.3 配置 Client 并启动 Wireshark

在 Client 上启动 Wireshark，并选择 enp0s3 接口监听网络流量。为尽可能完整地观察 ARP 和 DNS 解析过程，在访问网站前清除相关缓存：

Listing 9 清除 DNS 缓存

```
1 sudo ip neigh flush all
2 sudo resolvectl flush-caches
```

之后开始 Wireshark 抓包：

Listing 10 启动 Wireshark 抓包

```
1 sudo wireshark
```

在 Client 上先运行 Wireshark，选择监听以太网网卡 eth0 的所有流量；在 Client 上通过浏览器访问

<http://www.imool.com.cn>

，然后结束访问该网站；

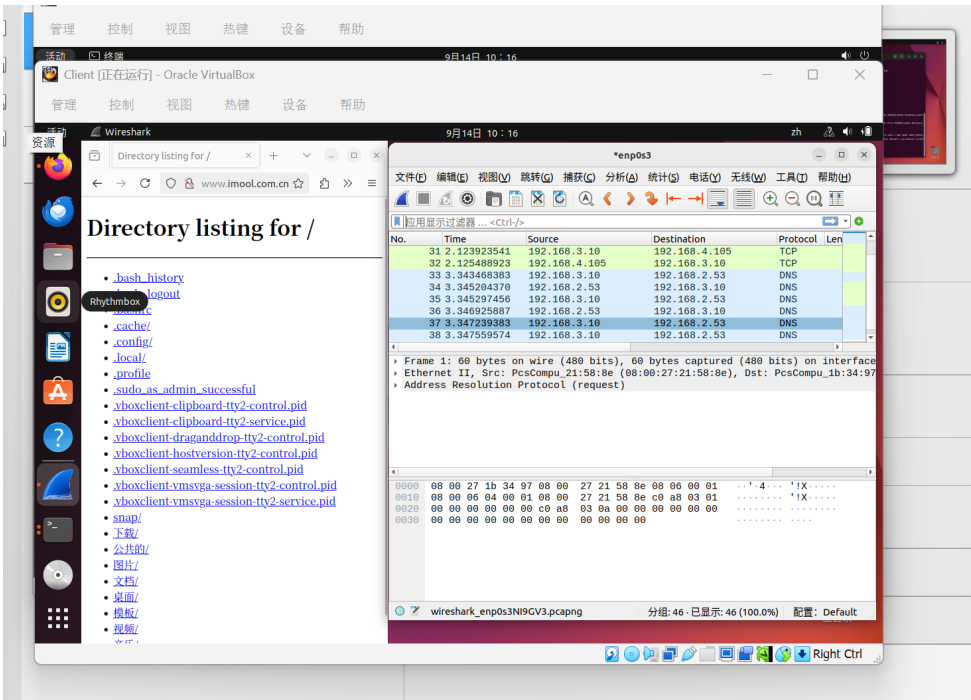


图 12 Wireshark 抓包截图

5.3.4 抓取相关的协议流量

抓取相关的 DNS 流量，如下：

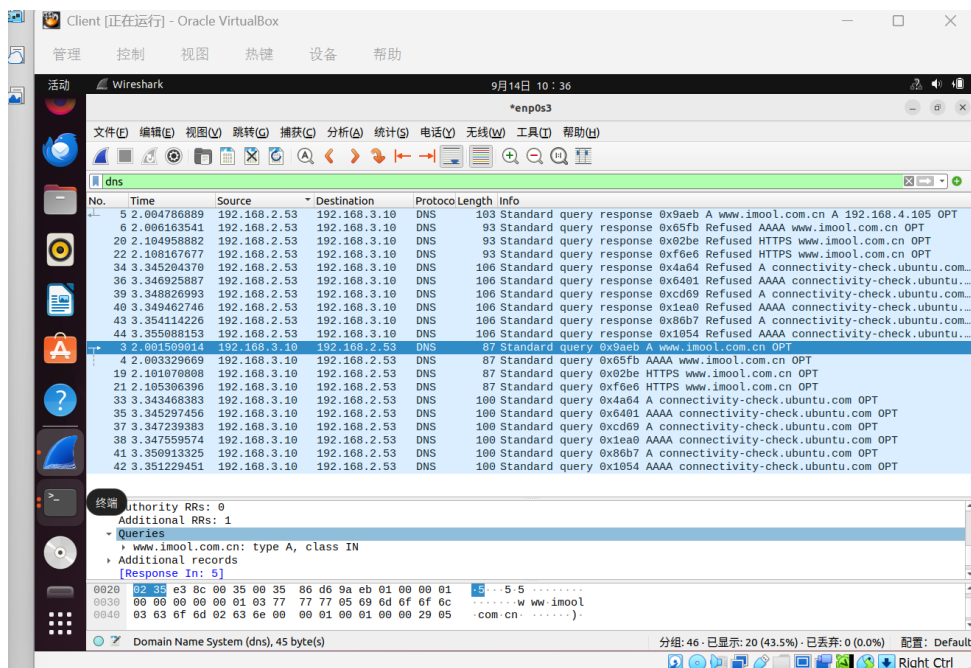


图 13 DNS 流量截图 1

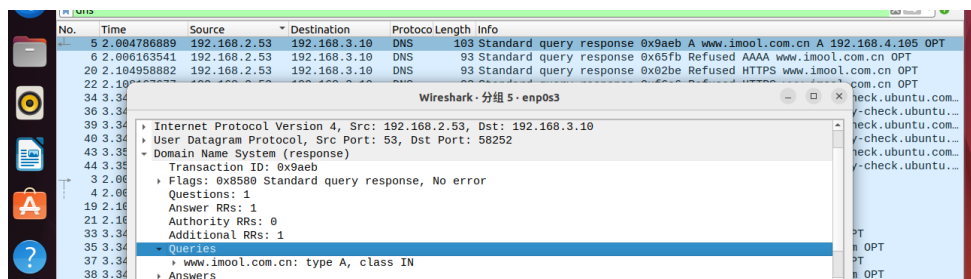


图 14 DNS 流量截图 2

抓取相关的 ARP 流量，如下：

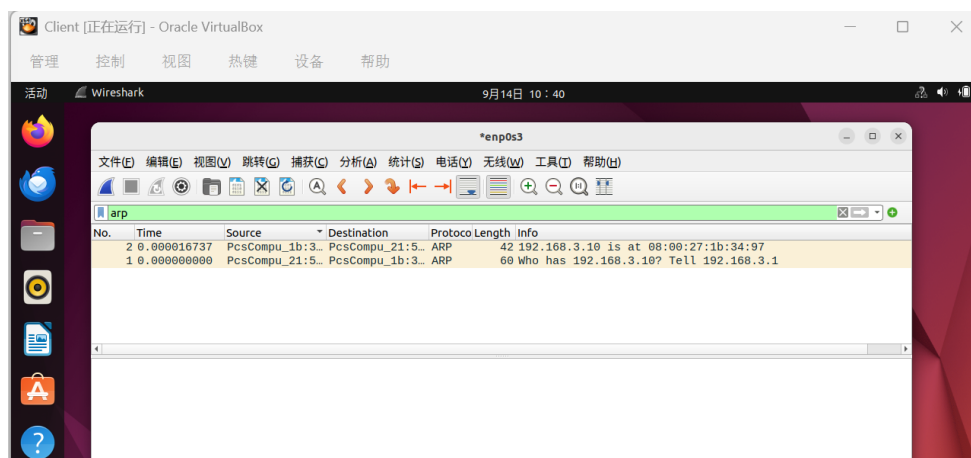


图 15 ARP 流量截图

抓取相关的 TCP 流量，如下：

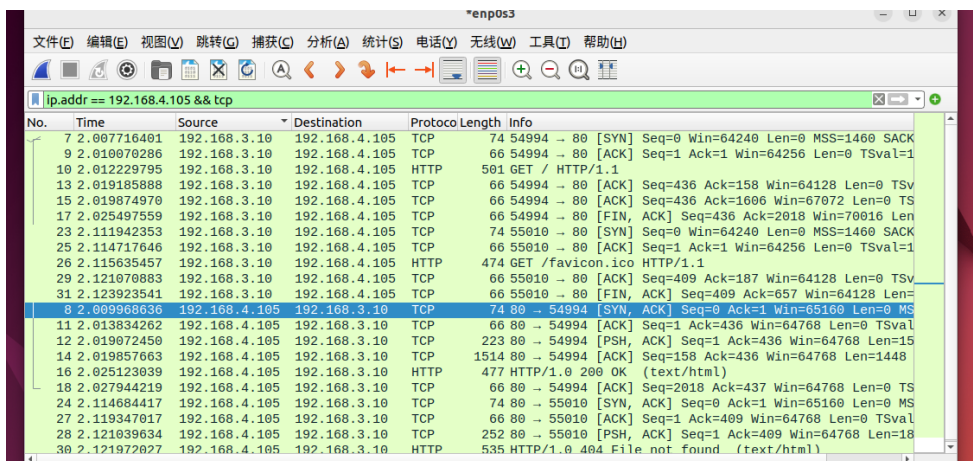


图 16 TCP 流量截图

抓取相关的 HTTP 流量，如下：

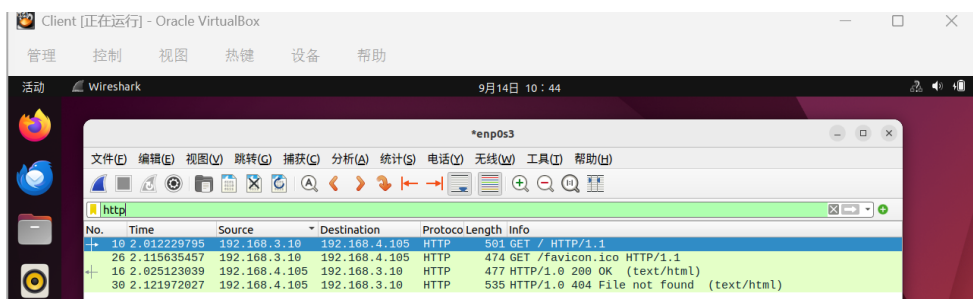


图 17 HTTP 流量截图 1

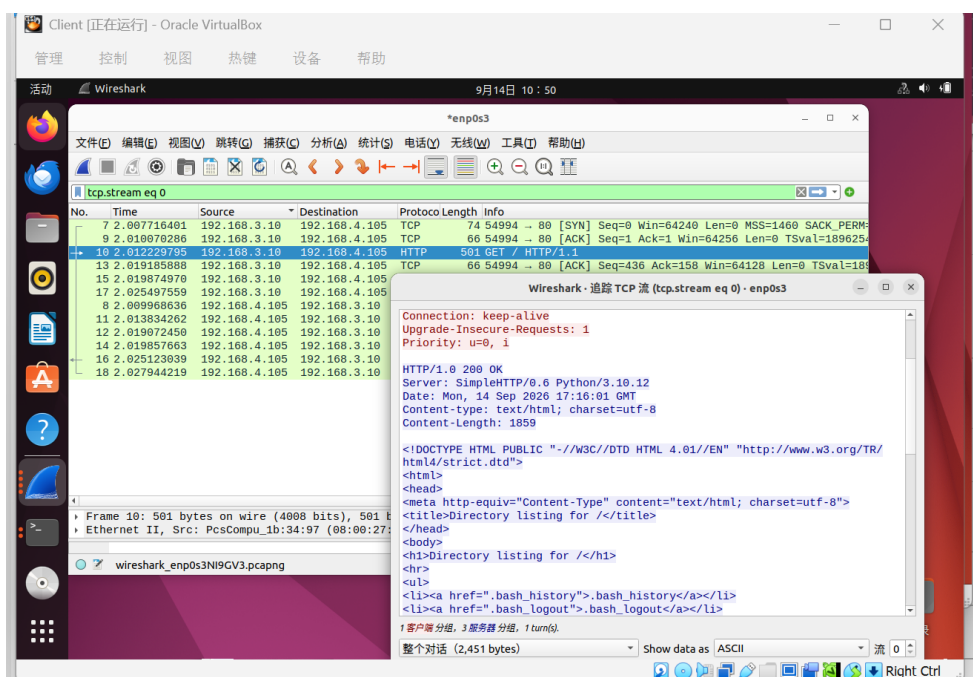


图 18 HTTP 流量截图 2

发现上述图片中没有 cookie，于是自己手写添加一个，创建一个简单的服务器：

Listing 11 创建简单的服务器

```
1 nano ~/cookie_server.py
```

写入：

Listing 12 写入简单的服务器

```
1 from http.server import HTTPServer, SimpleHTTPRequestHandler
2
3 class CookieHandler(SimpleHTTPRequestHandler):
4     def end_headers(self):
5         self.send_header("Set-Cookie", "username=wjy; Path=/")
6         super().end_headers()
7
8 server = HTTPServer(("0.0.0.0", 80), CookieHandler)
9
10 print("HTTP server running on port 80...")
11 server.serve_forever()
```

保存以后运行：

Listing 13 运行简单的服务器

```
1 sudo python3 ~/cookie_server.py
```

最后重新执行：wireshark 抓包，firebox 访问

http://www.imool.com.cn

，刷新页面，停止抓包，最后是找 HTTP，再使用追踪 TCP 流查看 cookie，如下图所示：

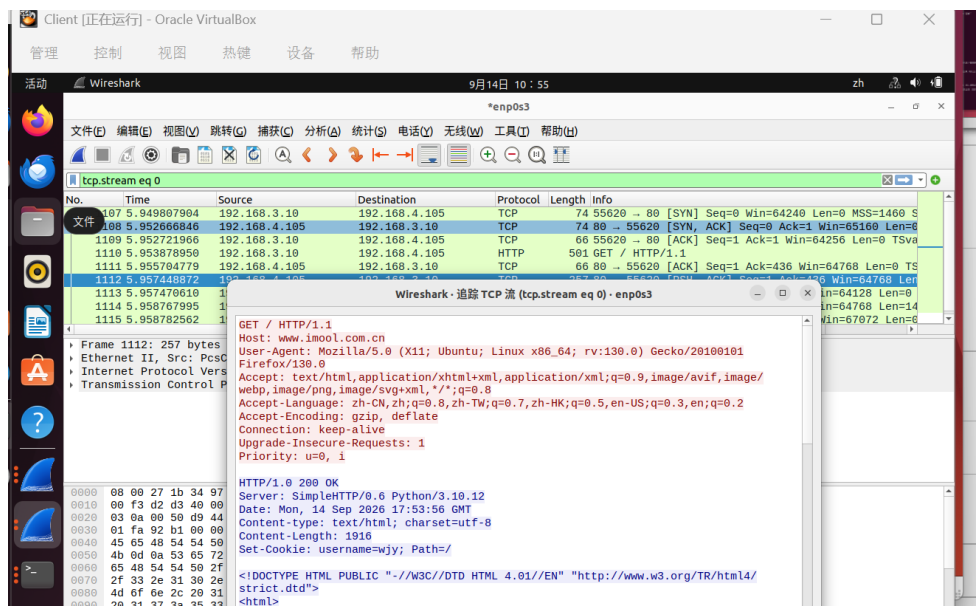


图 19 HTTP 流量截图 3

5.4 特别关注的相关问题

5.4.1 域名解析过程涉及哪些 IP 包，请求和响应分别是什么

虚拟机 Client（192.168.3.10）向 DNS Server（192.168.2.53）发送 DNS 查询请求，请求解析 www.imool.com.cn；DNS Server 返回请求的响应，将该域名解析为 Web Server 地址 192.168.4.105

5.4.2 ARP 解析过程中，网关的 MAC 地址是什么

根据 Wireshark 捕获到的 ARP Response（ARP 响应），默认网关 192.168.3.1 的 MAC 地址为：

08:00:27:1b:34:97

5.4.3 Client 与 www.imool.com.cn 的连接建立和拆除过程

TCP 连接建立采用三次握手：192.168.3.10 → 192.168.4.105 [SYN] 192.168.4.105 → 192.168.3.10 [SYN, ACK] 192.168.3.10 → 192.168.4.105 [ACK]

TCP 连接拆除采用四次挥手为：Client → Web Server FIN, ACK Web Server → Client ACK Web Server → Client FIN, ACK Client → Web Server ACK

5.4.4 使用 Wireshark 的协议流追踪功能提取 Cookie 信息

在 Wireshark 中选择 HTTP 数据流并使用 TCP Stream（TCP 流）追踪功能，可以观察到服务器返回的：

Listing 14 cookie 的显示

```
1 Set-Cookie: username=wjy; Path=/
```

其中 Cookie 名称为 username，值为 wjy，路径为 /。

5.5 实验小结

这一小节实验在实验一网络拓扑的基础上重新配置 DNS Server 和 Web Server，并利用 OpenWRT 实现不同子网之间的路由通信。通过 Wireshark 对访问 www.imool.com.cn 的全过程进行抓包，观察了 DNS 域名解析、ARP 地址解析、TCP 三次握手和四次挥手以及 HTTP 请求和响应过程，并通过 TCP 流追踪提取了 Cookie 信息。通过本实验，我们对浏览器访问一个 Web 网站过程中各层协议之间的配合关系有了更加直观的认识。

6 实验三：使用 Scapy 构造 ICMP Echo Request

6.1 实验目的

1. 学习和掌握 Scapy 工具的功能和使用方法，掌握 Scapy 构造包的流程；
2. 掌握 ICMP 包的结构，了解一个完整的 ICMP 包都需要满足哪些条件；
3. 实践网络数据包的伪造与发送，使用 Scapy 向受害者发送伪造的 ICMP 数据包。

6.2 实验配置

本实验使用 docker 搭建实验环境，Client(Attacker) 通过 OpenWRT 路由器与其他主机进行通信。

值得注意的是,docker 中的 ubuntu 是没有 GUI 界面的,但是我们可以通过远程桌面来实现

这个已经有现成的镜像实现了,直接 pull kasmweb/ubuntu-focal-desktop:1.16.0

详见附录中的 docker 环境配置,包括 dockerfile 也在里面,目录如下



图 20 实验配置目录

配置文件填写完之后，就可以开始部署

```
1 docker compose up -d
```

如果想要进入某个机器，就用

```
1 docker exec -it server bash
```

特别地, 对于 victim, 我们利用远程桌面来连接在主机浏览器访问 <https://localhost:6901>, 需要登录

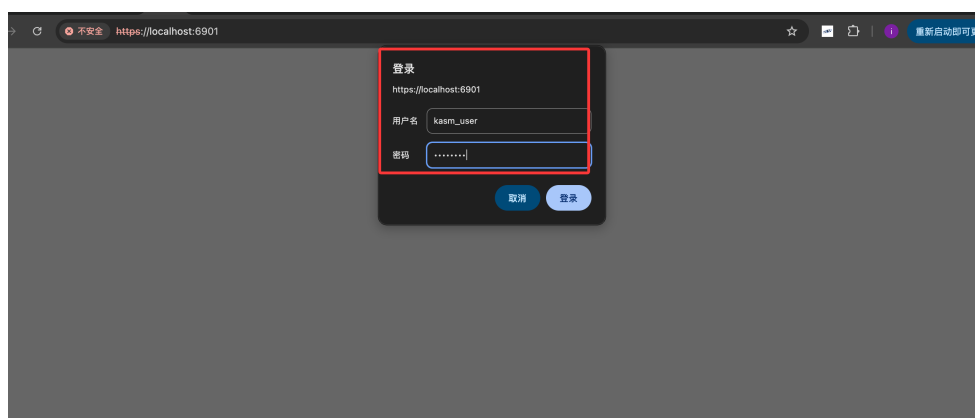


图 21 登录远程桌面

然后就就可以和其他虚拟机软件一样操控桌面端的 ubuntu 了, 使用体验差距不大, 比较方便的是直接同步了剪贴板

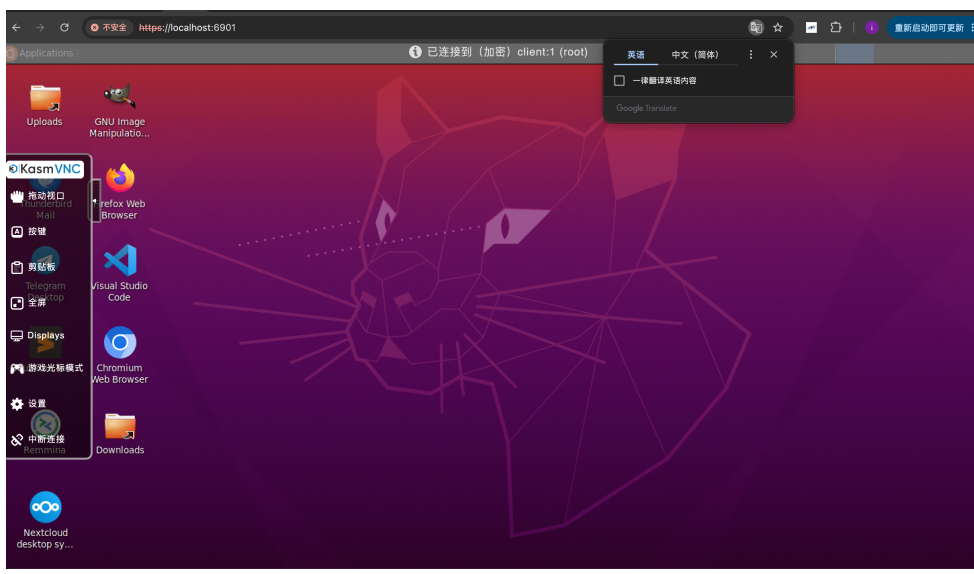


图 22 进入桌面

表 6 实验三各虚拟机网络接口及 IP 地址配置

虚拟机	Linux 内网卡名	IP 地址	默认网关
Victim(Client)	eth0	192.168.3.10/24	192.168.3.254
Attacker	eth0	192.168.2.100/24	192.168.2.1
User	eth0	192.168.4.105/24	192.168.4.1
OpenWRT	eth0	192.168.3.1/24	—
OpenWRT	eth1	192.168.4.1/24	—
OpenWRT	eth2	192.168.2.1/24	—

网络拓扑如图所示：

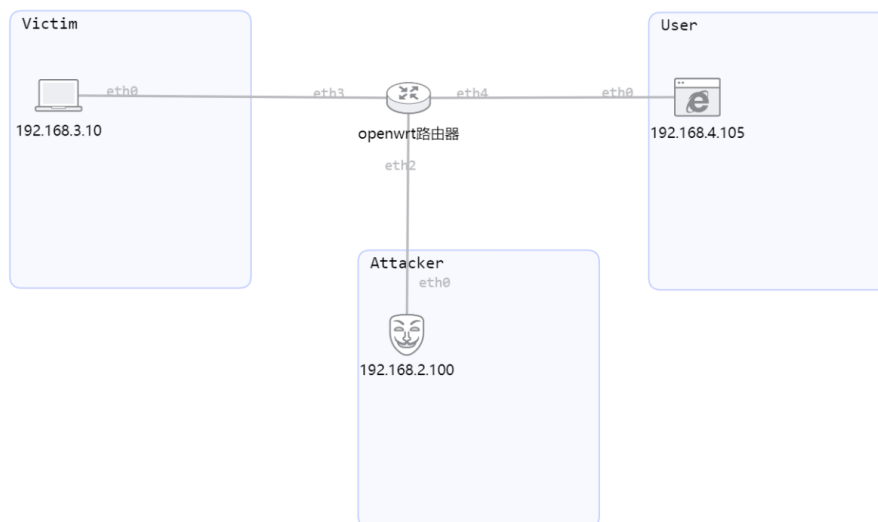


图 1-3-1 实验拓扑图

图 23 实验三网络拓扑图

6.3 实验过程

打开 scapy,



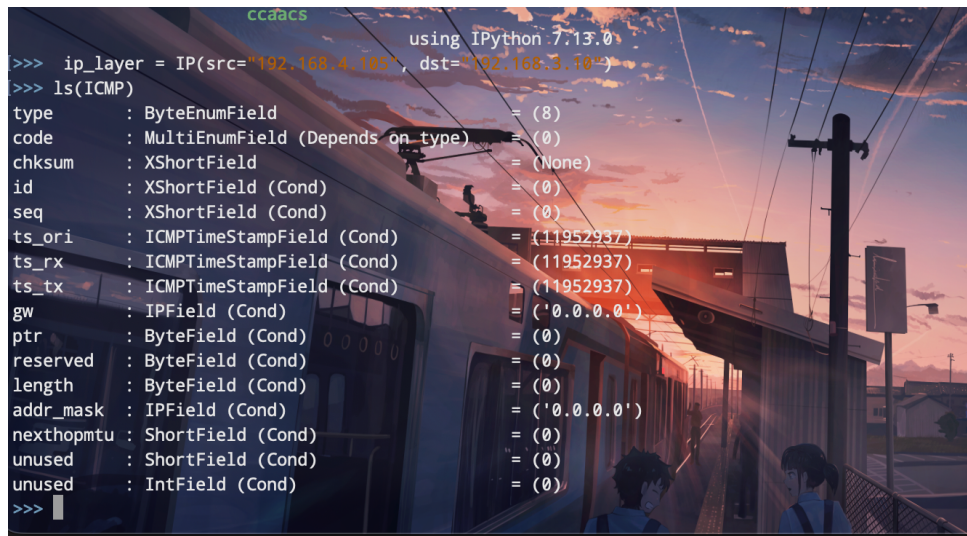
图 24 打开 scapy

6.3.1 创建 ip 层

将源 ip (src) 设置为 User ip, 将目的 ip (dst) 设置成 Victim ip;

```
1 ip_layer = IP(src="192.168.4.105", dst="192.168.3.10")
```

通过 ls(ICMP) 查看 ICMP 层默认字段值，其中 type=8 和 code=0 组合起来表示 ICMP Echo Request (Ping) 数据包，未设置的字段将使用其默认值填充；



```
ccaacs using IPython 7.13.0
>>> ip_layer = IP(src="192.168.4.105", dst="192.168.3.10")
>>> ls(ICMP)
type      : ByteEnumField              = (8)
code      : MultiEnumField (Depends on type) = (0)
checksum  : XShortField                = (None)
id        : XShortField (Cond)         = (0)
seq       : XShortField (Cond)         = (0)
ts_ori    : ICMPTimeStampField (Cond)  = (11952937)
ts_rx     : ICMPTimeStampField (Cond)  = (11952937)
ts_tx     : ICMPTimeStampField (Cond)  = (11952937)
gw        : IPField (Cond)             = ('0.0.0.0')
ptr       : ByteField (Cond)           = (0)
reserved  : ByteField (Cond)           = (0)
length    : ByteField (Cond)           = (0)
addr_mask : IPField (Cond)             = ('0.0.0.0')
nexthopmtu : ShortField (Cond)        = (0)
unused    : ShortField (Cond)         = (0)
unused    : IntField (Cond)           = (0)
>>>
```

图 25 创建 ip 层、查看 ICMP 层

6.3.2 创建 icmp 层、payload

```
1 icmp_layer = ICMP() #创建ICMP层
```

创建负载数据；

```
1 payload = "I come from Attacker-192.168.2.100"#载荷的内容
```

6.3.3 构造 ICMP Echo Request 数据包

将 IP 层、ICMP 层、负载层三者进行组合，生成一个 Ping 数据包；

```
1 icmp_request = ip_layer / icmp_layer / payload
```

通过.show() 方法来查看已构造好的数据包内容；

```
unused : IntField (Cond) = (0)
[>>> icmp_layer = ICMP()
[>>> payload = "I come from Attacker-192.168.2.100"
[>>> icmp_request = ip_layer / icmp_layer / payload
[>>> icmp_request.show()
###[ IP ]###
version= 4
ihl= None
tos= 0x0
len= None
id= 1
flags=
frag= 0
ttl= 64
proto= 1
chksum= None
src= 192.168.4.105
dst= 192.168.3.10
\options\
###[ ICMP ]###
type= echo-request
code= 0
chksum= None
id= 0x0
seq= 0x0
###[ Raw ]###
load= 'I come from Attacker-192.168.2.100'
```

图 26 创建 ICMP 层

6.3.4 发送数据包

```
[>>> send(icmp_request)
.
Sent 1 packets.
>>>
```

图 27 发送数据包

6.3.5 收到数据包

打开 wireshark, 然后监测 eth0 的流量, 注意要过滤掉 6901 端口的流量, 这个是用来远程桌面的

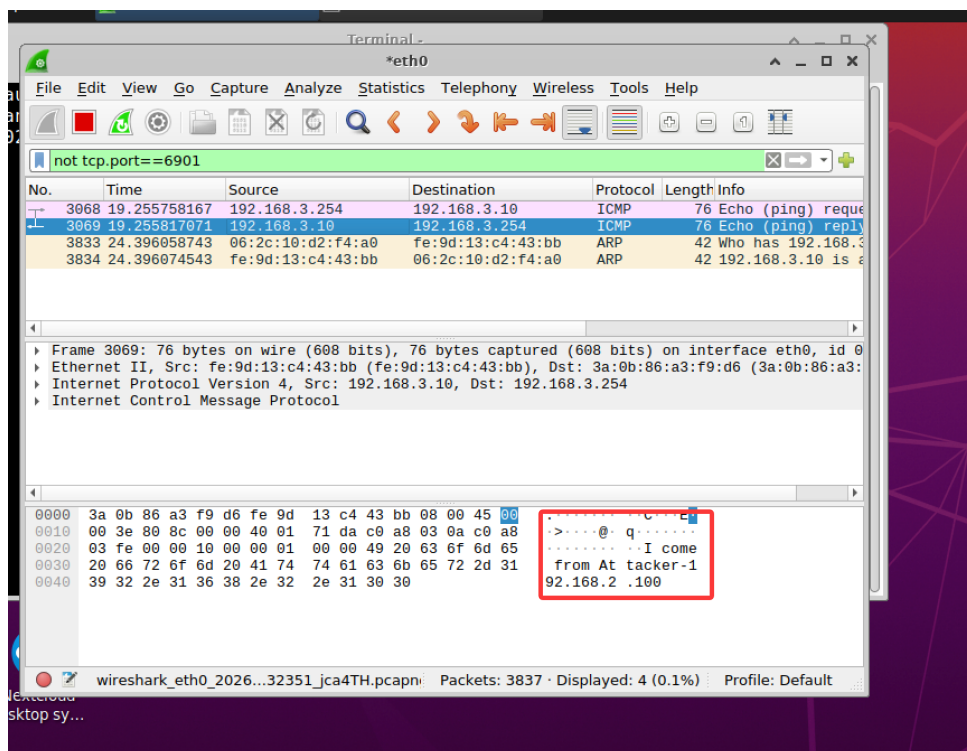


图 28 收到数据包

但是为什么这里是 192.168.3.254? 这是因为 Docker 的网络默认会打开 ip masquerade, 也就是 ip 伪装, 网关会将源 IP 地址替换为自身的地址 [1]



图 29 docker ip 伪装

有趣的是, 这正是我们这个实验要做的事, 不就是伪装吗? 我们需要在 docker-compose.yaml 里修改一下这个参数 (附录里是修改完的)



图 30 修改 docker-compose.yaml

改完再发送就成功了

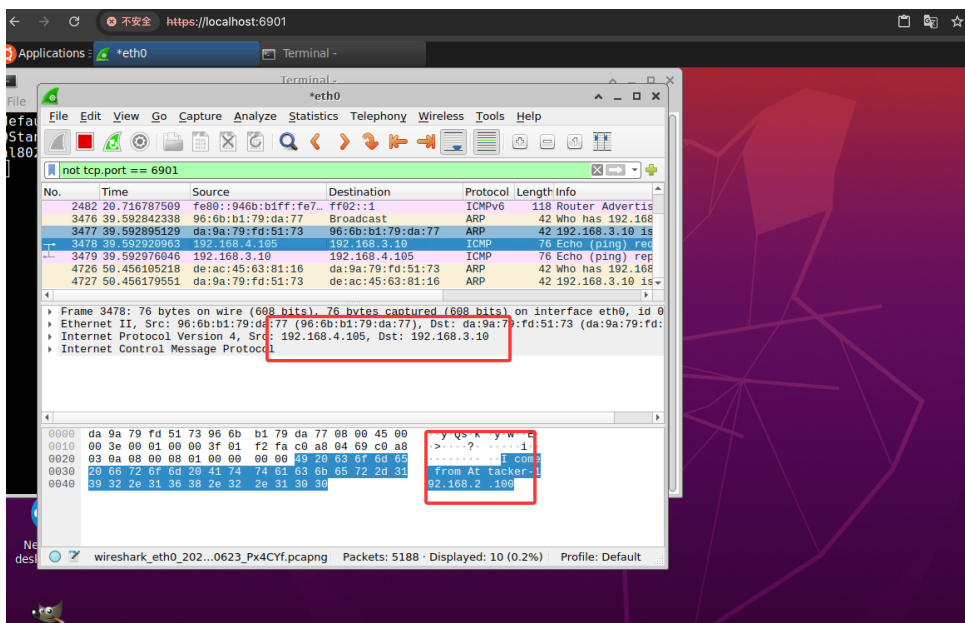


图 31 收到数据包

7 实验讨论

值得注意的是, 由于 openwrt 的版本不同, 实验指导书中的一些配置不再适用, 如 openwrt 防火墙的配置

```
1 list network 'lan1 lan2'
```

这里实验的时候发现总是 ping 不通, 新版 openwrt 会把 lan1 lan2 当成一个网络名称。

此时应该分成两行:

```
1 list network 'lan1'
2 list network 'lan2'
```

另一个值得注意的地方是, 实验指导书有些自相矛盾的地方, 实验 1 中要求 tcpdump 监测 eth0

因此，当使用 `sudo` 命令 `sudo su` 切换到 `root` 用户时，`sudo` 权限

(2) 用 `tcpdump` 监听端口 `eth0`，观察所有流量：

```
# tcpdump -i eth0 -n -e #-i: 指定要监听的网络接口。在这个例子中，eth0
接口的名称；-n: 禁用主机名、端口号等的名称解析。这样可以加快显示速度，并且
避免因解析导致的延迟；-e: 显示链路层头部信息，包括 MAC 地址。
```

图 32 `tcpdump` 监测 `eth0`

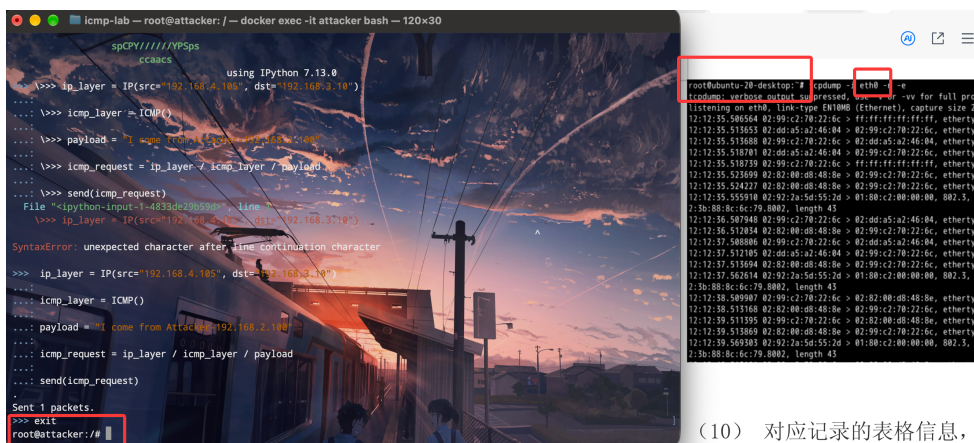
但是在比较新的 `ubuntu` 版本中，网卡不再以 `eth0` 等命名，而是 `enp0s3`，这在网络环境配置指导书里有

- 修改网络配置文件 `/etc/netplan/01-network-manager-all.yaml`，例如：

```
network:
  version: 2
  renderer: networkd
  ethernet:
    enp0s3:
      dhcp4: no
      addresses:
        - 192.168.10.2/24
      routes:
        - to: default
          via: 192.168.10.1
      nameservers:
        addresses:
          - 8.8.8.8
```

图 33 以 `enp0s3` 命名

可疑的是，实验指导书此处应当是使用 `docker` 做的，`docker` 中的 `ubuntu` 可以直接命名网卡，此处显然会对采用 `virtualbox` 方案的同学产生误导



(10) 对应记录的表格信息,

图 34 疑似 docker

8 参考资料

- [1] Docker. *Bridge Network Driver*. URL: <https://docs.docker.com/engine/network/drivers/bridge/> (visited on 09/19/2026).

A 附录：关键代码

A.0.1 docker-compose.yml

```
1  services:
2    route:
3      build:
4        context: ./openwrt
5      container_name: route
6      hostname: route
7      privileged: true
8      restart: unless-stopped
9
10   networks:
11     internal:
12       interface_name: eth0
13       ipv4_address: 192.168.3.1
14
15     external:
16       interface_name: eth1
17       ipv4_address: 192.168.4.1
18
19     dns_net:
20       interface_name: eth2
21       ipv4_address: 192.168.2.1
22   internal-server:
23     image: ubuntu:20.04
24     platform: linux/arm64
25     container_name: internal-server
26     hostname: internal-server
27
28     cap_add:
29       - NET_ADMIN
30       - NET_RAW
31
32   networks:
33     internal:
34       interface_name: eth0
35       ipv4_address: 192.168.3.200
36
```

```

37     command: >
38         bash -c "
39             apt-get update && DEBIAN_FRONTEND=noninteractive apt-get install -y
             iproute2 iputils-ping tcpdump net-tools && ip route del default && ip
             route add default via 192.168.3.1 && tail -f /dev/null
40         "
41
42 client:
43     build:
44         context: ./client
45
46     container_name: client
47     hostname: client
48
49     privileged: true
50     shm_size: "512m"
51
52     ports:
53         - "6901:6901"
54
55     environment:
56         VNC_PW: password
57
58     networks:
59         internal:
60             interface_name: eth0
61             ipv4_address: 192.168.3.10
62
63
64 dns:
65     build:
66         context: ./dns
67     container_name: dns
68     hostname: dns
69
70     cap_add:
71         - NET_ADMIN
72         - NET_RAW
73
74     depends_on:

```

```

75     - route
76
77     networks:
78         dns_net:
79             interface_name: eth0
80             ipv4_address: 192.168.2.53
81
82     command:
83         - /bin/bash
84         - -c
85         - |
86             ip route del default || true
87             ip route add default via 192.168.2.1 dev eth0
88             exec dnsmasq --no-daemon
89     attacker:
90         build:
91             context: ./attacker
92         container_name: attacker
93         hostname: attacker
94
95         cap_add:
96             - NET_ADMIN
97             - NET_RAW
98
99     depends_on:
100         - route
101
102     networks:
103         dns_net:
104             interface_name: eth0
105             ipv4_address: 192.168.2.100
106
107     command:
108         - /bin/bash
109         - -c
110         - |
111             ip route del default || true
112             ip route add default via 192.168.2.1 dev eth0
113             tail -f /dev/null
114

```

```
115
116 server:
117     build:
118         context: ./server
119     container_name: server
120     hostname: server
121
122     cap_add:
123         - NET_ADMIN
124         - NET_RAW
125
126     depends_on:
127         - route
128
129     networks:
130         external:
131             interface_name: eth0
132             ipv4_address: 192.168.4.105
133
134     command:
135         - /bin/bash
136         - -c
137         - |
138             ip route del default || true
139             ip route add default via 192.168.4.1 dev eth0
140             exec nginx -g 'daemon off;'
141
142
143 networks:
144     internal:
145         driver: bridge
146         ipam:
147             config:
148                 - subnet: 192.168.3.0/24
149                   gateway: 192.168.3.254
150
151     external:
152         driver: bridge
153         driver_opts:
154             com.docker.network.bridge.enable_ip_masquerade: "false"
```

```

155     ipam:
156         config:
157             - subnet: 192.168.4.0/24
158               gateway: 192.168.4.254
159
160     dns_net:
161         driver: bridge
162         ipam:
163             config:
164                 - subnet: 192.168.2.0/24
165                   gateway: 192.168.2.254

```

A.0.2 Dockerfile

openwrt

```

1 FROM scratch
2 ADD openwrt-armv8-rootfs.tar.gz /
3 CMD ["/sbin/init"]

```

client(victim)

```

1 FROM kasmweb/ubuntu-focal-desktop:1.16.0
2
3 USER root
4
5 ENV DEBIAN_FRONTEND=noninteractive
6
7 RUN apt-get update && \
8     apt-get install -y \
9         wireshark \
10        tcpdump \
11        iproute2 \
12        iputils-ping \
13        net-tools \
14        curl \
15        vim \
16        netcat-openbsd && \
17     rm -rf /var/lib/apt/lists/*

```

attacker

```

1 FROM ubuntu:20.04
2 ENV DEBIAN_FRONTEND=noninteractive
3 RUN apt-get update && \
4     apt-get install -y \
5         iproute2 \
6         iputils-ping \
7         net-tools \
8         scapy \
9         tcpdump \
10                                vim \
11         curl && \
12     rm -rf /var/lib/apt/lists/*

```

B 附录：江李阳个人实验记录

B.1 实验二：使用 Wireshark 分析 Web 访问过程流量

本附录记录个人完成实验二时的实际操作和抓包结果。实验目标网址按照本组实验约定使用 <http://www.seclab.online>。由于本次实验的网络配置经过调整，下面的环境表以本次截图中实际使用的地址为准，与实验一最终采用的网络配置并不完全相同。

B.1.1 实验环境

表 7 实验二个人实验环境

项目	实际配置或说明
客户端 (Attacker)	Ubuntu, 网卡 ens33 , IP 为 192.168.3.10/24, 默认网关为 192.168.3.1
DNS 服务器	192.168.2.53, 客户端通过 UDP 53 端口进行域名查询
OpenWRT	内网接口地址为 192.168.3.1; 配置截图中还使用了 DNS 侧地址 192.168.2.1
External Server	IP 为 192.168.4.105, 提供本次 HTTP 访问服务
抓包工具	Wireshark, 捕获接口为客户端的 ens33
访问目标	http://www.seclab.online

B.1.2 实验过程

首先在客户端启动 Wireshark，选择 ens33 网卡进行抓包，然后访问目标网址。启动抓包时客户端会产生一些系统背景流量，实际分析时使用显示过滤器缩小到目标 DNS、ARP 和 HTTP 连接。客户端启动 Wireshark 的情况如图35所示。

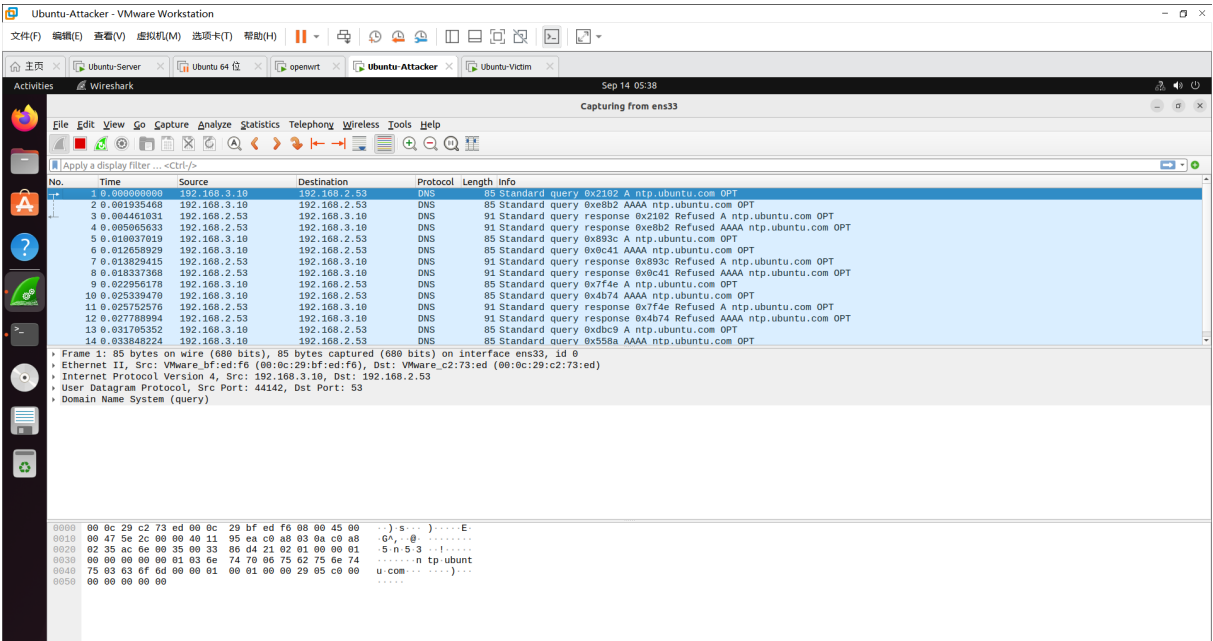


图 35 客户端在 ens33 网卡上启动 Wireshark

在访问目标网址之前，客户端需要解析域名。使用下面的过滤器：

```
dns.qry.name=="www.seclab.online"
```

可以观察到客户端 192.168.3.10 向 DNS 服务器 192.168.2.53 发出查询，并收到 www.seclab.online 的 A 记录 192.168.4.105。由此得到 Web 服务器的 IP 地址，后续 TCP 连接的目标就是该地址。

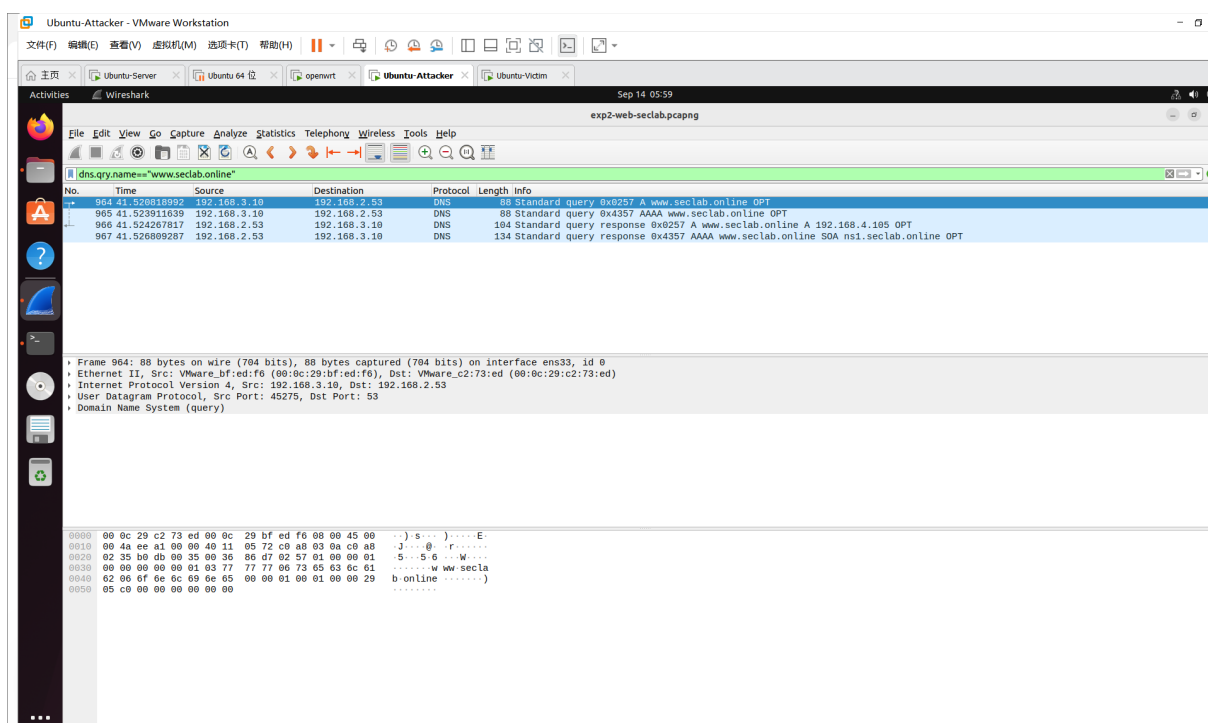


图 36 www.seclab.online 的 DNS 查询与响应

清空或重新触发 ARP 缓存后，客户端还需要获得默认网关的 MAC 地址。抓包结果显示网关 192.168.3.1 对应的 MAC 地址为 00:0c:29:c2:73:ed，客户端网卡的 MAC 地址为 00:0c:29:bf:ed:f6。这说明客户端访问其他网段的服务器时，二层下一跳是 OpenWRT，而不是直接使用 External Server 的 MAC 地址。

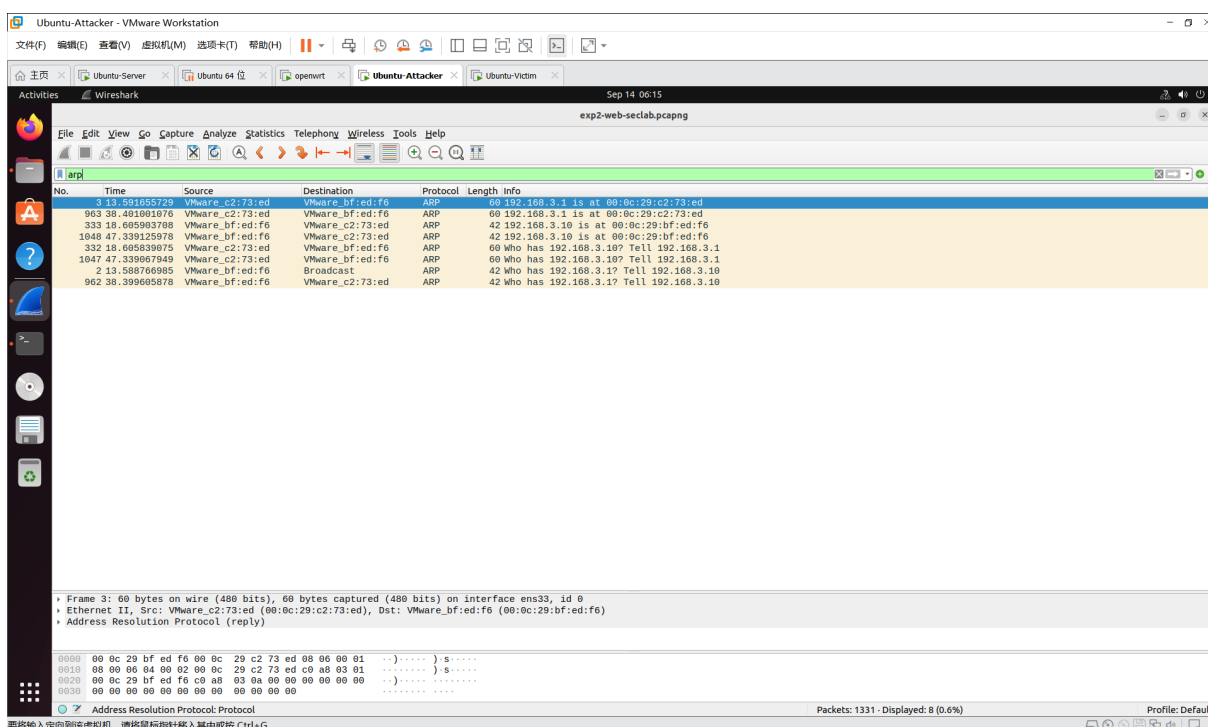


图 37 客户端与默认网关之间的 ARP 报文

随后使用过滤器 `ip.addr == 192.168.4.105 and tcp.port == 80` 查看客户端与 Web 服务器之间的 TCP 和 HTTP 报文。抓包中可以观察到客户端发送 SYN、服务器返回 SYN/ACK、客户端再发送 ACK，三次握手完成后开始传输 HTTP 数据。之后还能看到客户端的 HTTP GET 请求以及服务器返回的 302、200 等 HTTP 响应，最后连接通过 FIN/ACK 进行关闭。

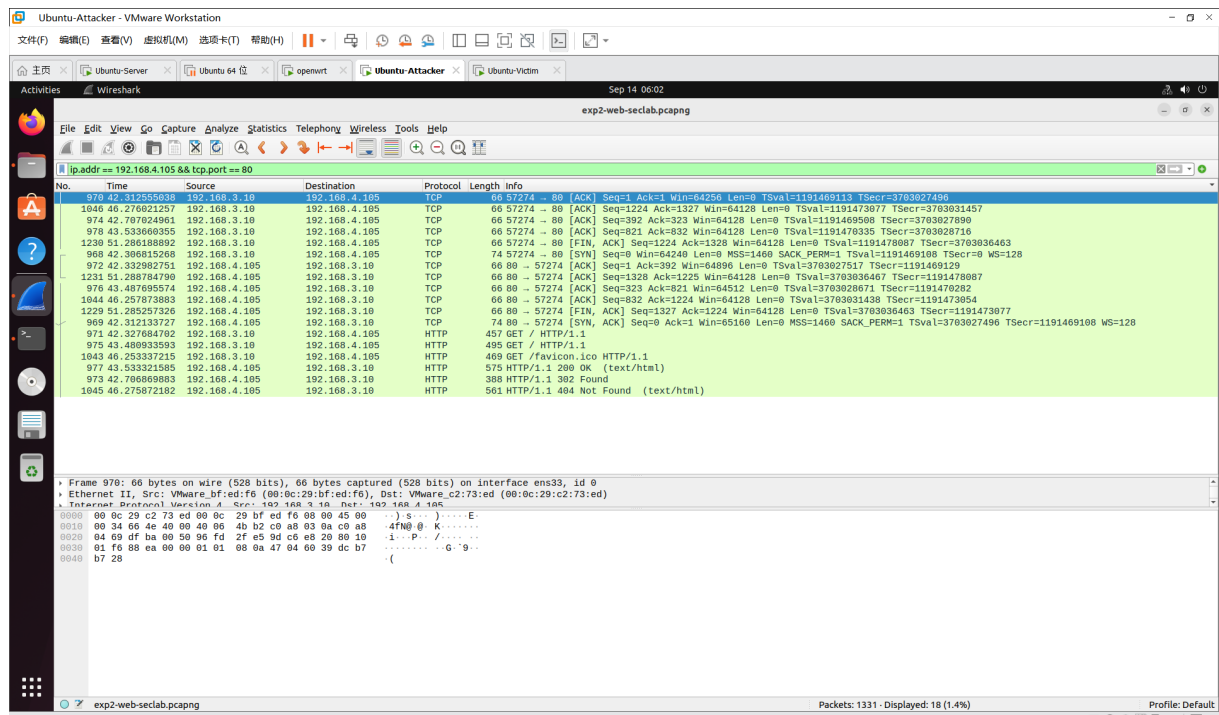


图 38 客户端与 Web 服务器之间的 TCP 和 HTTP 报文

在 Wireshark 中选中 HTTP 数据包并使用“Follow TCP Stream”，可以还原本次 HTTP 会话的请求和响应。服务器在响应中设置了 `lab_session=exp2-client-demo` Cookie，客户端后续请求又通过 Cookie 字段携带了相同的值，说明 Cookie 在 HTTP 会话中的设置和回传过程可以直接从明文 HTTP 流中观察到。

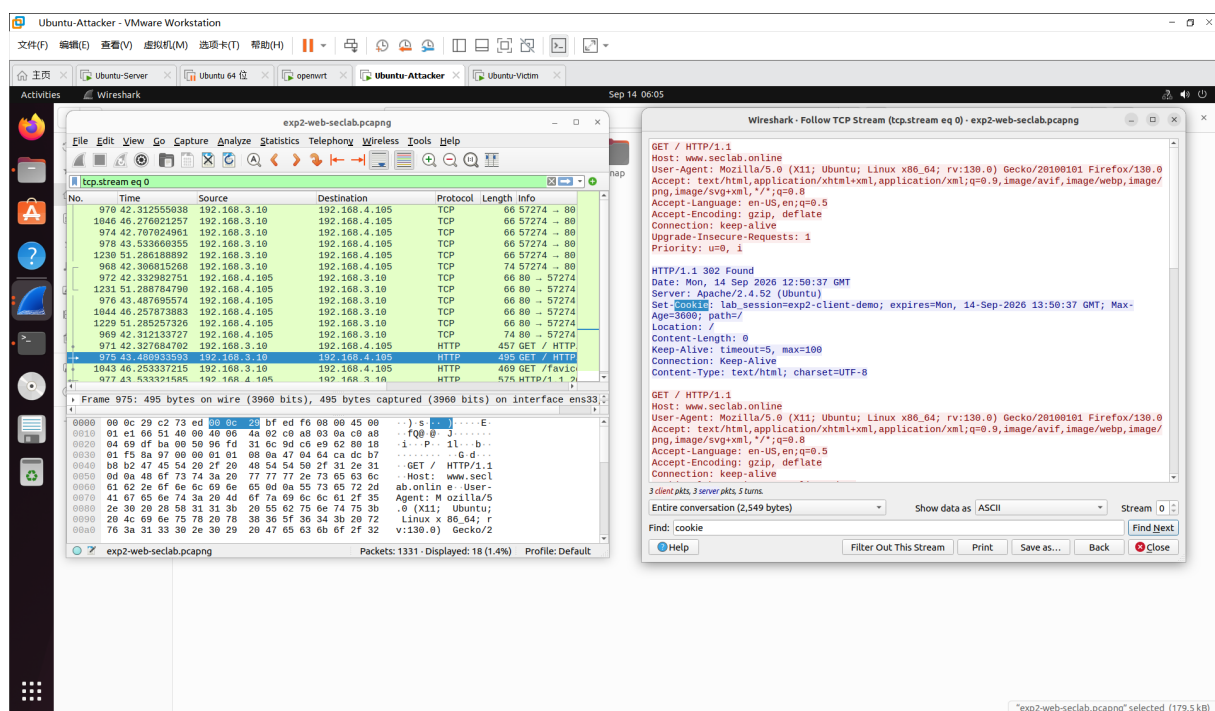


图 39 Follow TCP Stream 中的 HTTP 请求、响应与 Cookie

B.1.3 实验结果

本次抓包结果对应关系如下：

表 8 实验二主要报文分析结果

报文或阶段	抓包结果
DNS	192.168.3.10 查询 192.168.2.53, www.seclab.online 的 A 记录为 192.168.4.105
ARP	客户端解析默认网关 192.168.3.1 的 MAC 地址
TCP	192.168.3.10 与 192.168.4.105:80 完成三次握手
HTTP	客户端发送 GET, 服务器返回 302、200 等响应
Cookie	服务器设置 lab_session=exp2-client-demo, 客户端后续请求携带该 Cookie

由此可以看出，一次 Web 访问并不是单独的一条 HTTP 报文，而是先经过 ARP 获得下一跳地址，再通过 DNS 将域名解析为 IP 地址，然后建立 TCP 连接，最后在连接上传输 HTTP 请求和响应。Wireshark 的协议过滤和 Follow TCP Stream 功能可以把这些不同层次的报文关联起来。

B.2 实验三：使用 Scapy 构造 ICMP Echo Request

本附录记录个人使用 Scapy 构造并发送 ICMP Echo Request 的过程。该次截图记录采用了与实验一不同的地址配置，并且按照实验指导书的要求使用 External Server 的地址作为伪造源地址，因此环境信息和报文中的源、目的地址需要分别说明。

B.2.1 实验环境

表 9 实验三个人实验环境

项目	实际配置或说明
Attacker	Ubuntu, 网卡 ens33 , 实际 IP 为 192.168.2.100/24, 默认网关为 192.168.2.1
Victim	Ubuntu, 网卡 ens33 , 实际 IP 为 192.168.3.10/24, 默认网关为 192.168.3.1
伪造源地址	192.168.4.105; 该地址只写入 IP 首部, 并不是 Attacker 网卡的实际地址
工具	Scapy 2.4.4, 以 root 权限启动
负载	I come from Attacker-192.168.2.100

B.2.2 数据包构造与发送

首先在 Attacker 上使用 root 权限启动 Scapy。实验中先构造 IP 层, 将源地址设置为 192.168.4.105、目的地址设置为 Victim 的 192.168.3.10; 再构造默认 ICMP 层和自定义负载, 最后使用 “/” 运算符将三层内容组合为一个数据包。

Listing 15 以 root 权限启动 Scapy

```
1 sudo scapy
```

Listing 16 使用 Scapy 构造 ICMP Echo Request

```
1 ip_layer = IP(src="192.168.4.105", dst="192.168.3.10")
2 icmp_layer = ICMP()
3 payload = "I come from Attacker-192.168.2.100"
4 icmp_request = ip_layer / icmp_layer / payload
5 icmp_request.show()
6 send(icmp_request)
```

Scapy 中 ICMP() 使用默认字段值, 其中 **type=8** 表示 Echo Request, **code=0** 表示该请求没有错误子类型。调用 **show()** 后, 可以看到 IP 层的源地址为 192.168.4.105、目的地址为 192.168.3.10, ICMP 层为 Echo Request, Raw 层中保存了自定义字符串。

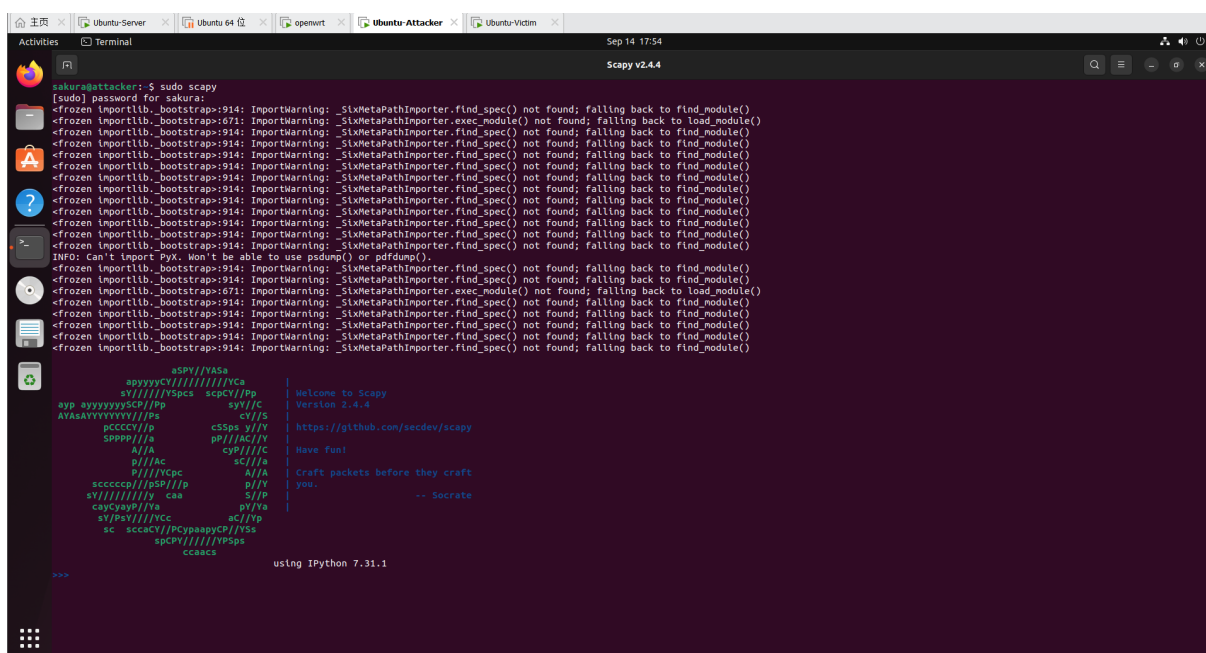


图 40 Attacker 以 root 权限启动 Scapy

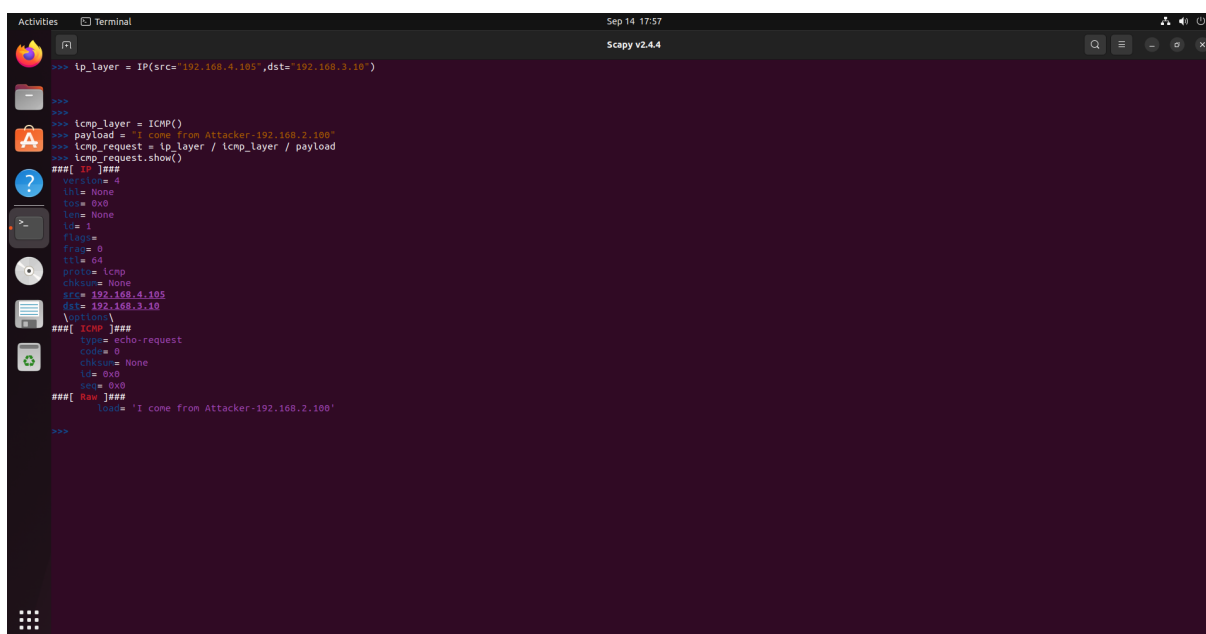


图 41 Scapy 构造的 ICMP Echo Request 及 show 输出

调用 `send(icmp_request)` 后，Scapy 显示 “Sent 1 packets.”，表示已经发送一个数据包。由于这里使用的是 `send()`，实际二层封装和下一跳选择由系统路由完成；本次实验的重点是观察 Scapy 构造的 IP、ICMP 和 Raw 数据内容。

B.2.3 Victim 侧验证

在 Victim 侧使用 `tcpdump` 监听 `ens33`，过滤 ICMP 报文。抓包结果显示：

- 收到的 ICMP Echo Request 的源 IP 为 192.168.4.105，目的 IP 为 192.168.3.10；
- Raw 负载中可以看到 “I come from Attacker-192.168.2.100”；
- Victim 返回源 IP 为 192.168.3.10、目的 IP 为 192.168.4.105 的 ICMP Echo Reply。

因此，Victim 能够根据 ICMP 的请求类型进行响应，同时抓包结果也证明了 Attacker 成功将伪造的源地址和自定义负载放入了发送的 IP 数据包中。

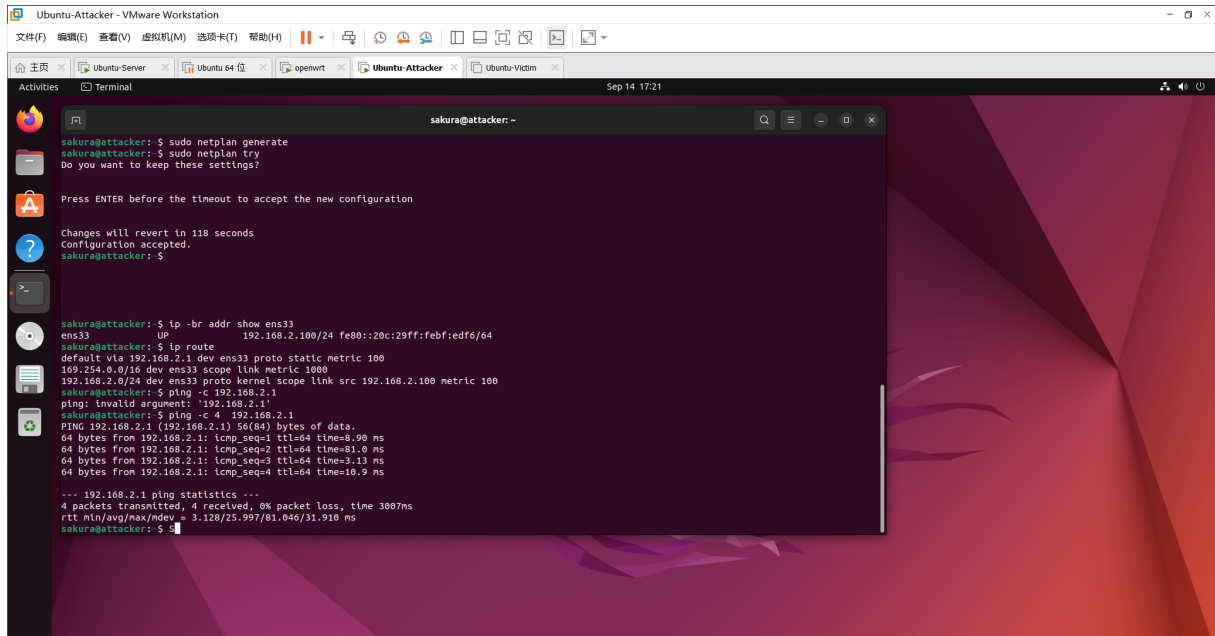


图 42 Attacker 的地址配置及连通性验证

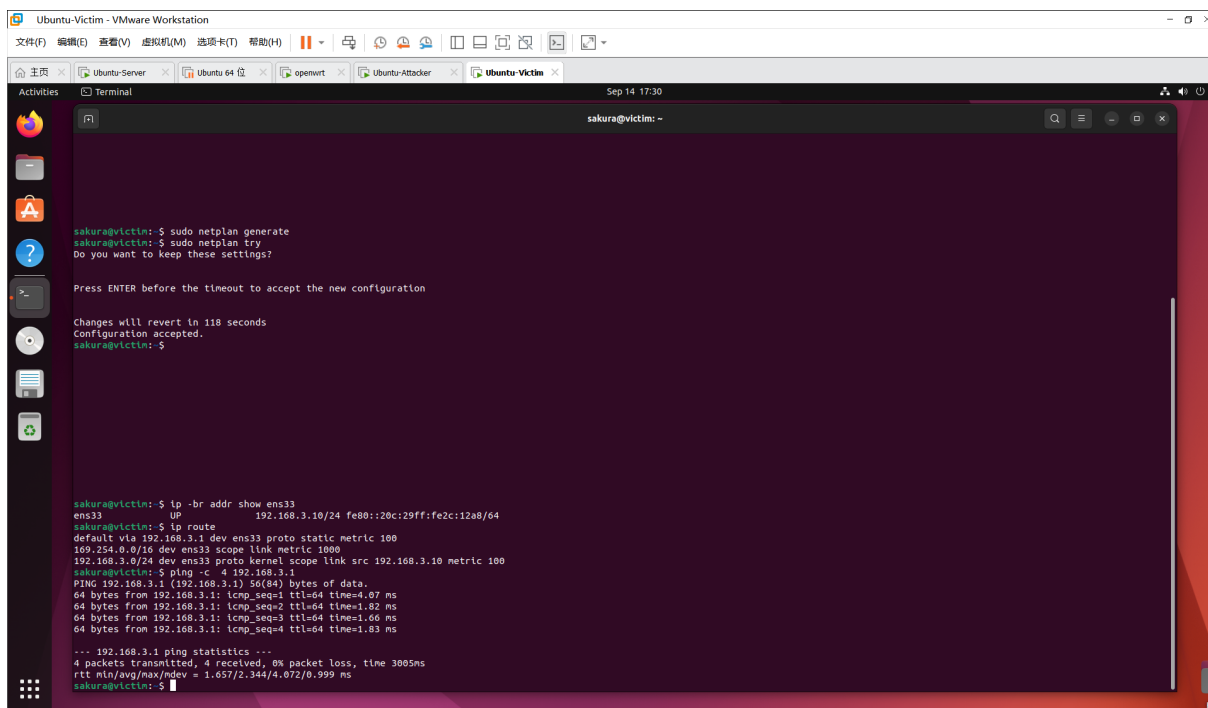


图 43 Victim 的地址配置及网关连通性验证

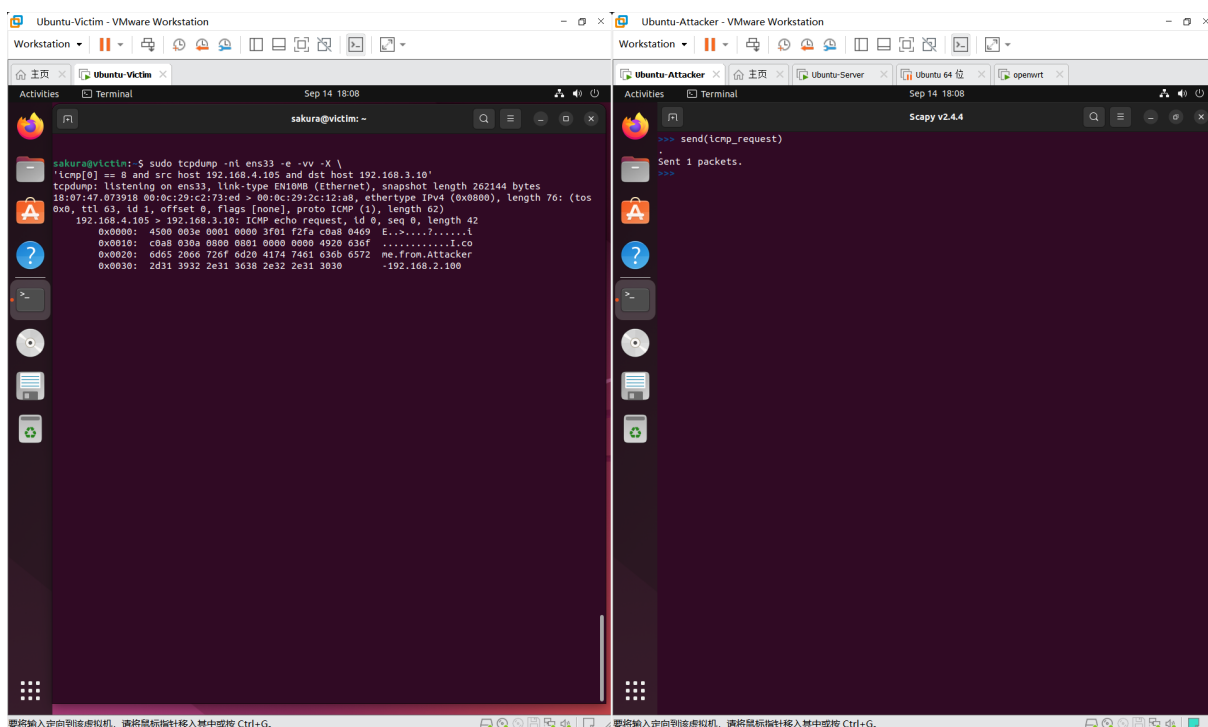


图 44 Scapy 发送数据包及 Victim 侧 tcpdump 抓包结果

B.2.4 实验结果

本次实验成功构造并发送了一个 ICMP Echo Request。需要注意的是，Attacker 的真实地址是 192.168.2.100，而报文 IP 首部中的源地址是 192.168.4.105，因此该实验展示的

是带有源 IP 伪造的 ICMP 报文，而不是普通的 Attacker 到 Victim 的 ping。Victim 侧收到请求并返回 Echo Reply，说明报文构造、发送以及抓包验证均完成。